9장
데이터 분석 기초

지금까지는 리스트나 딕셔너리에 임의로 데이터를 넣어 프로그래밍했습니다. 실 데이터는 아니었지만, 입력해야 해서 조금 번거로웠죠. 그래서 미리 정제되어 있는 데이터를 읽는 방법을 배워 보겠습니다. 그리고 읽어 들인 파일로 데이터 분석합니다. 데이터 파일을 읽어 들이는 방법을 알면 다양한 데이터를 활용할 수 있고, 데이터를 통계적 알고리즘에 적용해 보는 것도 가능합니다. 이번 장에서는 간단히 파일 입출력 방법과 실제 데이터를 사용하므로 더욱 재미있을 겁니다. 그럼 시작해 볼까요?

9.1

데이터 준비: 파일에서 데이터 읽어 들이기

9.1.1 CSV 파일이란

데이터 분석에서 사용하는 데이터 파일은 여러 종류가 있습니다. XLS, XLSX 형식의 엑셀 파일도 있고, 텍스트로만 이루어진 TXT 파일도 있습니다. 여기서는 CSV 파일을 다루겠습니다. CSV(Comma Separated Values) 파일은 쉼표(,)로 구분된 텍스트 형식의 파일로, 메모장이나 엑셀 등에서 열 수 있습니다.

TIP 이 책에서 CSV 파일을 사용하는 이유는 엑셀 파일보다 용량이 적기 때문입니다. 그리고 엑셀 파일은 지금까지 배우지 않은 다른 라이브러리와 데이터 타입을 알아야 하기 때문에 지금 단계로는 조금 어려운 측면이 있습니다. 그래서 지금까지 배운 리스트와 딕셔너리만으로 충분히 다룰 수 있는 CSV 파일을 사용합니다.

먼저 이 책의 자료실에서 card.csv 파일을 내려받습니다. 그런 다음, 내려받은 파일을 선택한 상태에서 [마우스 오른쪽 버튼 클릭 → 연결 프로그램 → 메모장]을 눌러 메모장으로 파일을 엽니다. 그러면 다음처럼 쉼표(,)로 구분된 텍스트 파일이 열립니다.



▲ 그림 9-1 메모장으로 CSV 파일 열기

이 파일은 2019년 10월부터 12월까지 3개월간의 실제 카드 이용내역입니다. 수많은 텍스트가 행마다 쉼표(,)로 구분되어 있는데, 내용을 보기가 조금 불편하죠.

card.csv 파일을 메모장이 아닌 엑셀로 열어 봅시다. 앞에서와 마찬가지로 파일을 선택한 상태에서 [마우스 오른쪽 버튼 클릭 → 연결 프로그램 → Excel]을 눌러 파일을 엽니다(엑셀 프로그램이 없는 경우 열어 보지 않아도 됩니다).



▲ 그림 9-2 엑셀로 CSV 파일 열기

카드 이용내역이 표 형태로 나옵니다. 조금 더 보기 편하죠? 파일에서 첫 번째 행은 표의 헤더(머리글 행)에 해당하고, 두 번째 행부터 실제 데이터가 되는 카드 이용내역입니다. 각 행은 한 건의 카드 이용내역 정보를 담고 있습니다. 메모장에서 쉼표로 구분한 각 행의 텍스트는 칸으로 구분됩니다. 헤더를 보면 첫 번째 열(A열)이 이용일시, 두 번째 열(B열)이 승인번호, 마지막 열(I열)이 매입상태입니다.

결국 CSV 파일은 행과 열이 있는 표 형식의 데이터를 표현한 것임을 알 수 있습니다. 이런 행과 열이 있는 데이터 타입을 앞 장에서 배운 적이 있죠? 바로 이차원 리스트입니다. 그래서 파이썬 프로그램에서는 CSV 파일을 읽은 후, 이차원 리스트로 데이터 타입을 변경해 데이터 분석을 진행합니다.

9.1.2 CSV 파일 읽어 들이기

파이썬 프로그램에서 CSV 파일을 어떻게 읽어 들이는지 알아봅시다. 먼저 파이썬 파일 하나를 만듭니다. 그리고 앞에서 내려받은 card.csv 파일을 방금 만든 파이썬 파일과 같은 폴더에 넣습니다. CSV 파일과 파이썬 파일을 다른 폴더에 두면 CSV 파일을 읽을 때 따로 경로를 설정해야 합니다. 따라서 이 책에서는 같은 폴더에 넣고 진행하겠습니다. 그리고 파이썬 파일에 이후 내용을 작성합니다.

CSV 파일은 [파일 열기 → 파일 읽기 → 표의 헤더 제거하기 → 리스트로 변환하기]의 4단계로 읽어 들입니다.

CSV 파일을 다루려면 `csv` 라이브러리가 필요합니다. 따라서 가장 먼저 `import csv`로 작성하는 파이썬 파일에 라이브러리를 추가해야 합니다. 같은 파일 내에서는 한 번만 포함하면 되므로 이후 실행할 때는 셀마다 넣지 않아도 됩니다.

첫 번째 단계인 파일 열기는 `f = open('파일명.csv')`로 실행합니다. 이 코드는 CSV 파일을 열고 파일 내용을 변수 `f`에 담는다는 의미입니다. 일단 여기까지만 작성하고 `print()`로 변수 `f`에 담긴 값을 출력해 봅시다.

```
import csv

f = open('card.csv')
print(f)
```

```
실행결과
<_io.TextIOWrapper name='card.csv' mode='r' encoding='cp949'>
```

NOTE 코랩에서 파일 열기

코랩에서는 파일을 열려면 다음과 같이 파일 업로드 명령어를 추가해야 합니다.

```
import csv
from google.colab import files
uploaded = files.upload()

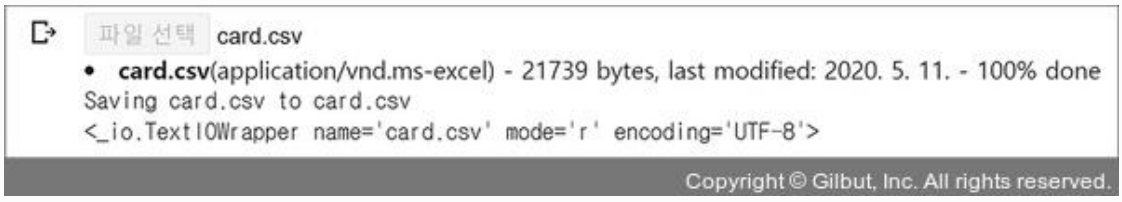
f = open('card.csv')
print(f)
```

두 줄을 추가하고 실행하면 다음처럼 [파일 선택] 버튼이 나옵니다. [파일 선택] 버튼을 누르고 card.csv 파일을 선택합니다.



▲ 그림 9-3 코랩에 파일 올리기

파일 업로드가 끝나면 다음처럼 `100% done` 표시가 뜹니다. 이후부터는 주피터 노트북과 동일하게 CSV 파일을 읽어들일 수 있습니다.



▲ 그림 9-4 코랩에 파일 올리기 완료

card.csv 파일에는 많은 데이터가 들어 있는데 겨우 한 줄만 출력됩니다. 이는 `open()`만으로는 파일의 데이터를 읽지 못한다는 뜻입니다. 화면에 나온 실행결과는 card.csv 파일의 정보만 보여 준 것이죠.

여기서 주목할 부분은 마지막의 `encoding`입니다. 사용하는 운영체제가 Windows라면 `encoding`을 `utf8`로, macOS라면 `cp949`로 설정해야 합니다(같은 파일이더라도 주피터 노트북과 코랩에서 인코딩 방식이 다르게 나타날 수 있습니다). 이럴 때는 다음처럼 `open()` 명령어에 `encoding` 옵션을 추가합니다.

```
# 운영체제가 Windows인 경우 encoding = 'utf8', macOS인 경우 encoding = 'cp949'
f = open('card.csv', encoding='utf8')
print(f)
```

```
실행결과
<_io.TextIOWrapper name='card.csv' mode='r' encoding='utf8'>
```

옵션을 추가하니 출력 결과도 `encoding='utf8'`로 바뀝니다. 이 부분이 PC의 운영체제와 맞게 설정돼야 다음 단계에서 오류가 발생하지 않습니다.

두 번째 단계인 파일 읽기는 명령어 `csv.reader()`에 CSV 파일의 내용을 저장한 변수 `f`를 넣습니다. 그리고 이를 다시 변수 `data`에 저장합니다. 여기까지 진행한 후 `data`에 저장된 값을 출력해 봅시다.

```
f = open('card.csv', encoding='utf8')
data = csv.reader(f)
print(data)
```

```
실행결과
<_csv.reader object at 0x00000221F0F93E48>
```

이번에도 기대한 데이터는 출력되지 않고 알아보기 어려운 실행결과가 한 줄 출력됩니다. 사실 CSV 파일의 데이터를 `data` 변수에 저장하긴 했지만, 결과에 출력할 수 있는 데이터 타입이 아니라서 그렇습니다(실행결과의 내용은 `csv.reader()`를 실행한 결과가 담긴 메모리 위치를 나타냅니다).

> TIP `UnicodeDecodeError`가 발생했다면 `open()`의 `encoding` 옵션이 운영체제에 맞게 설정됐는지 확인해 주세요. 운영체제에 맞춰 바꿔도 오류가 발생한다면 `euc-kr`로 설정하거나 `encoding` 옵션을 아예 없애고 실행해 보기 바랍니다. 옵션이 없어야만 오류가 발생하지 않는 경우도 있습니다.

여전히 `data` 변수에 담긴 내용을 보려면 한 단계를 더 거쳐야 합니다. 네 번째 단계인 리스트로 변환하기로, `data` 변수의 데이터 타입을 데이터 분석에서 사용할 수 있는 리스트로 변경합니다. 데이터 타입을 리스트로 바꿀 때는 6장에서 배웠듯이 `list()`로 감싸면 됩니다.

```
f = open('card.csv', encoding='utf8')
data = csv.reader(f)
data = list(data)
print(data)
```

```
실행결과
[['이용일시', '승인번호', '본인구분', '브랜드', '이용카드', '가맹점명', '이용금액', '이용구분',
'매입상태'], ['2019-10-12 9:13', '1972753', '본', 'S&', '185', '네이버파이낸셜(주)', '546800',
'일시불(A)', '부분취소']… ]
```

드디어 원하던 데이터가 출력됩니다. 가장 앞에 대괄호 2개는 리스트 안에 리스트, 즉 이차원 리스트라는 의미임을 알죠? 그런데 실행결과 앞부분에 표의 헤더 부분도 함께 출력됩니다. 헤더는 데이터 분석에 필요한 순수한 데이터가 아닙니다. 그래서 헤더를 제거해야 합니다. 이는 앞의 형식에 나왔듯이 `next()` 명령어로 처리합니다. `next(data)`는 읽어온 `data`에서 한 행을 건너뛰고 다음 행을 읽게 합니다. 이 부분은 `data`를 리스트로 변환하기 전인 세 번째 단계에서 먼저 수행해야 합니다.

```
f = open('card.csv', encoding='utf8')
data = csv.reader(f)
next(data)
data = list(data)
print(data)
```

실행결과

```
[['2019-10-12 9:13', '1972753', '본', 'S&', '185', '네이버파이낸셜(주)', '546800', '일시불(A)',
'부분취소'], ['2019-10-12 11:56', '3399357', '본', 'S&', '185', 'GS샵', '60440', '일시불(A)',
'전표매입']…]
```

헤더 부분이 없어지고 온전히 데이터만 출력됐습니다. 실행결과는 뒤에서 더 자세히 살펴보겠습니다. 여기서는 CSV 파일을 읽어 들이는 4단계 [파일 열기 → 파일 읽기 → 표의 헤더 제거하기 → 리스트로 변환하기]와 명령어 형식만 잘 기억해 두세요.

CSV 파일 읽어 들이기 형식

```
import csv                # csv 라이브러리 포함하기

f = open('파일명.csv')    # 파일 열기
data = csv.reader(f)      # 파일 읽기
next(data)                # 표의 헤더 제거하기
data = list(data)         # 리스트로 변환하기
```

9.2

데이터 추출: 필요한 데이터만 골라내기

파일 데이터가 `data` 변수에 저장된 상태를 전제하고 내용을 진행하므로 파일 읽어 들이기 4단계가 실행된 상태인지 확인해 주세요. 이후로도 앞의 코드가 실행 완료된 상태라 가정하고 진행합니다.

9.2.1 원하는 데이터 추출하기

card.csv 파일을 읽고 리스트로 변환해 `data`에 저장했습니다. 실행결과를 보면 대괄호 2개가 있는 이차원 리스트였죠. `data`의 타입을 출력해 이차원 리스트가 맞는지 확인해 봅시다.

```
print(type(data))
```

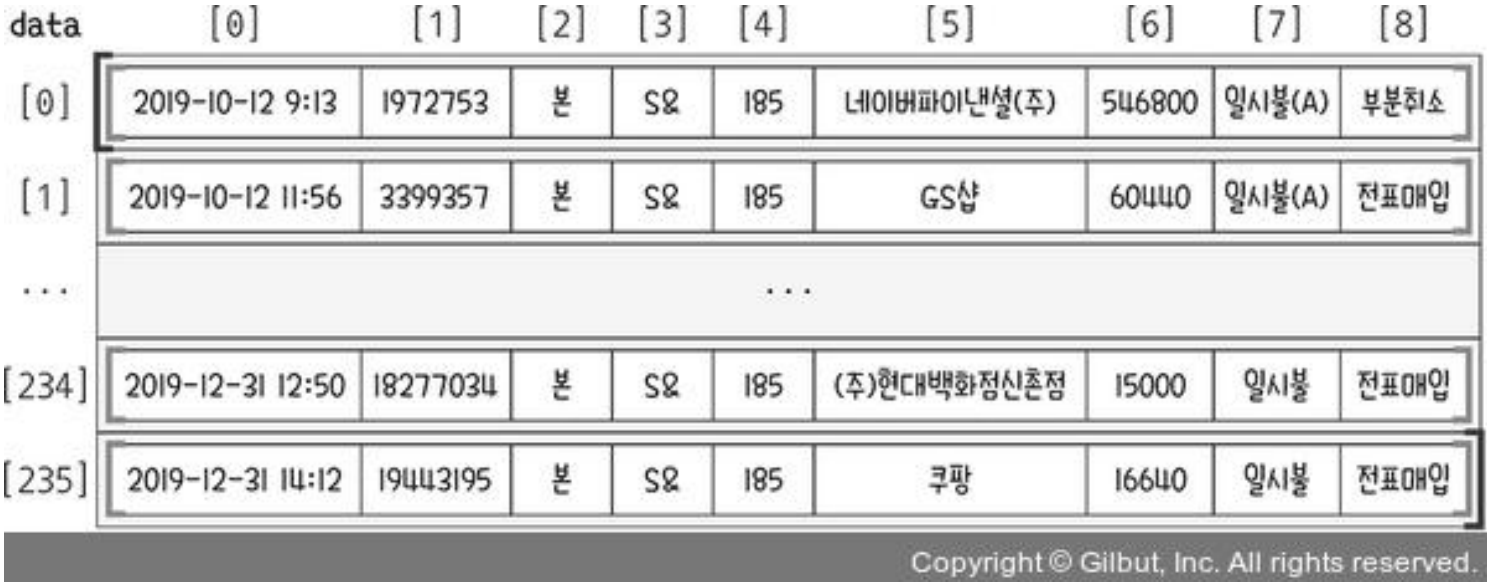
```
실행결과
<class 'list'>
```

리스트 타입이라고 나옵니다. `data`의 길이도 구해 봅시다.

```
print(len(data))
```

```
실행결과
236
```

이차원 리스트의 길이를 구하면 총 행의 개수가 출력됩니다. `data`에는 236개 행이 있군요. 이용내역이 236개라는 의미이며, 리스트의 인덱스가 `[0]`부터 시작해 `[235]`까지 있다는 뜻입니다. 이런 이차원 리스트 구조를 그려 보면 다음과 같습니다.



▲ 그림 9-5 이차원 리스트 data의 구조

이차원 리스트 `data` 안에는 236개의 일차원 리스트가 있습니다. 카드 이용내역 한 건이 일차원 리스트 1개로 표현됩니다. 첫 번째 이용내역(2019-10-12 9:13)이 `data`의 첫 번째 일차원 리스트이므로 `data[0]`이 되고, 두 번째 이용내역(2019-10-12 11:56)이 두 번째 일차원 리스트이므로 `data[1]`이 됩니다.

한 행 출력하기
이번에는 `data`의 첫 행을 출력해 봅시다.

```
print(data[0])
```

```
실행결과
['2019-10-12 9:13', '1972753', '본', 'S&', '185', '네이버파이낸셜(주)', '546800', '일시불(A)',
'부분취소']
```

첫 행은 2019년 10월 12일에 네이버파이낸셜(주)에서 546,800원을 결제한 건입니다.

여러 행 한꺼번에 출력하기
이번에는 처음부터 세 번째 이용내역까지 출력해 봅시다. 인덱스 [0]부터 [2]까지 출력하면 됩니다. 이때 슬라이싱을 사용해 `data[0:3]` 또는 0을 빼고 `data[:3]`으로 작성하면 되겠죠?

```
print(data[:3])
```

```
실행결과
[['2019-10-12 9:13', '1972753', '본', 'S&', '185', '네이버파이낸셜(주)', '546800', '일시불(A)',
'부분취소'], ['2019-10-12 11:56', '3399357', '본', 'S&', '185', 'GS샵', '60440', '일시불(A)',
'전표매입'], ['2019-10-13 9:16', '12621902', '본', 'S&', '185', 'G마켓', '91520', 'P(A)', '전표
매입']]
```

여러 건을 한 번에 출력하면 이처럼 결과가 이차원 리스트로 나옵니다. CSV 파일을 열어서 가장 위에 있는 세 건의 이용내역이 맞는지 직접 확인해 보세요.

특정 값 골라서 출력하기

그럼 첫 번째 이용내역의 이용금액만 출력해 봅시다. 첫 번째 이용내역은 `data[0]`입니다. 그리고 이용금액은 `data[0]` 리스트의 인덱스 [6]에 있죠. 따라서 `data[0][6]`을 출력하면 첫 번째 이용내역의 이용금액을 알 수 있습니다.



▲ 그림 9-6 첫 번째 이용내역의 이용금액

```
print(data[0][6])
```

```
실행결과
546800
```

첫 번째 이용내역의 가맹점명과 이용금액을 한꺼번에 출력해 봅시다. 앞의 코드 설명을 이해했다면 쉽게 작성할 수 있을 겁니다.

```
print(data[0][5], data[0][6])
```

```
실행결과
네이버파이낸셜(주) 546800
```

가맹점명은 열 인덱스 [5]에 있죠. 따라서 `data[0][5]`와 `data[0][6]`를 연결해 출력하면 됩니다. 이번에는 이용일시까지 출력해 봅시다.

```
print(data[0][0], data[0][5], data[0][6])
```

```
실행결과
2019-10-12 9:13 네이버파이낸셜(주) 546800
```

이용일시는 첫 열에 있으므로 열 인덱스 [0]를 사용하면 됩니다.

9.2.2 제어구조로 원하는 데이터 추출하기

앞에서 반복문과 조건문을 활용하면 이차원 리스트를 다루기가 쉬웠습니다. 수많은 데이터가 들어 있는 데이터 파일에서도 반복문과 조건문을 사용하면 전체 데이터에서 원하는 데이터를 추출하기도 쉽고 코드도 덜 복잡해집니다. 이번에는 반복문을 활용해 카드 이용내역에서 원하는 데이터를 뽑아 보겠습니다.

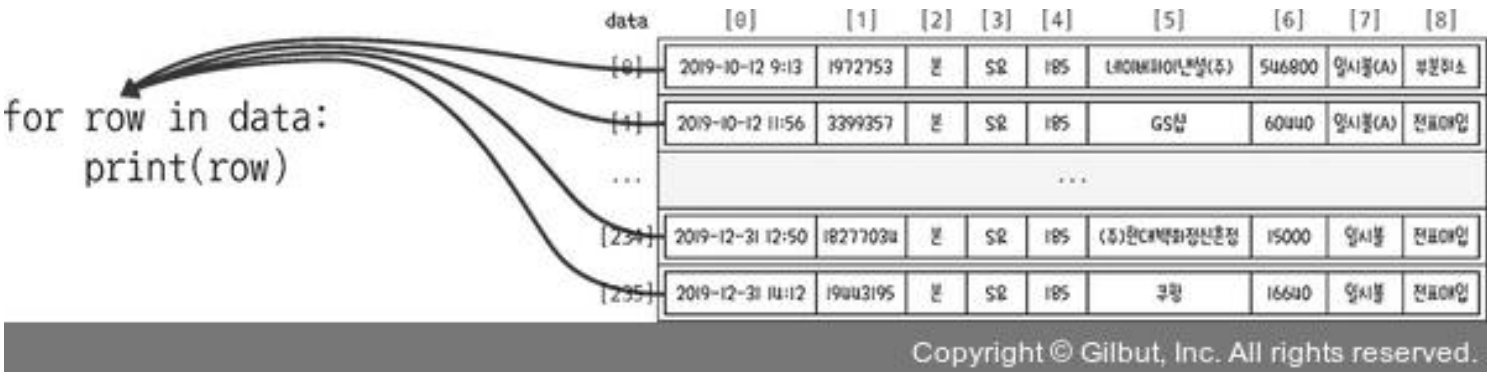
반복문으로 데이터 추출하기

현재 카드 이용내역은 변수 `data`에 이차원 리스트 형태로 저장되어 있습니다. `data`에서 한 행은 한 건의 이용내역입니다. 따라서 반복문의 반복 구간에 이차원 리스트를 넣으면 한 행(일차원 리스트)씩 변수에 저장됩니다. 이를 이용해 다음과 같이 이용내역을 한 건씩 출력할 수 있습니다.

```
for row in data:
    print(row)
```

```
실행결과
['2019-10-12 9:13', '1972753', '본', 'S&', '185', '네이버파이낸셜(주)', '546800', '일시불(A)', '부분취소']
['2019-10-12 11:56', '3399357', '본', 'S&', '185', 'GS샵', '60440', '일시불(A)', '전표매입']
…
```

반복문 내부에 `print(row)`가 있어서 반복할 때마다 `row`에 담긴 데이터가 출력됩니다(행을 뜻하는 `row`를 변수로 사용). 따라서 반복할 때마다 이용내역이 한 건씩 출력되고 반복이 끝나면 모든 이용내역이 출력됩니다. 반복 회차별로 `row`에 담기는 데이터는 다음과 같습니다.



Copyright © Gilbut, Inc. All rights reserved.

반복 회차	row에 담기는 데이터
1번째	data[0] ['2019-10-12 9:13', '1972753', '본', 'S2', '185', '네이버파이낸셜(주)', '546800', '일시불(A)', '부분취소']
2번째	data[1] ['2019-10-12 11:56', '3399357', '본', 'S2', '185', 'GS샵', '60440', '일시불(A)', '전표매입']
...	
235번째	data[234] ['2019-12-31 12:50', '18277034', '본', 'S2', '185', '(주)현대백화점신촌점', '15000', '일시불', '전표매입']
236번째	data[235] ['2019-12-31 14:12', '19443195', '본', 'S2', '185', '쿠팡', '16640', '일시불', '전표매입']

Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 9-7 반복문의 실행과정

그럼 모든 이용내역에서 이용금액만 출력하려면 어떻게 해야 할까요? 그림에서 보듯이 `row`에는 이용내역 한 건에 해당하는 일차원 리스트가 들어갑니다. 이용금액은 열 인덱스 [6]에 있죠. 따라서 반복문 `for row in data`에서 `row[6]`을 출력하면 전체 이용내역을 돌며 인덱스 [6]에 위치한 각 행(일차원 리스트)의 일곱 번째 요소를 출력합니다. 실행해 봅시다.

```
for row in data:
    print(row[6])
```

```
실행결과
546800
60440
91520
...
```

이용금액만 출력됩니다. CSV 파일을 직접 열어서 금액이 맞는지 확인해 보세요.

이번에는 반복문을 활용해 전체 카드 이용내역의 이용일시, 가맹점명, 이용금액을 차례대로 출력해 봅시다. 이용일시는 열 인덱스 [0]에, 가맹점명은 [5]에, 이용금액은 [6]에 있습니다.

```
for row in data:
    print(row[0], row[5], row[6])
```

```
실행결과
2019-10-12 9:13 네이버파이낸셜(주) 546800
2019-10-12 11:56 GS샵 60440
2019-10-13 9:16 G마켓 91520
...
```

각 카드 이용내역에서 이용일시, 가맹점명, 이용금액만 골라서 출력됩니다. 가맹점명과 이용금액을 입력할 때 인덱스를 뒤에서부터 세는 것이 더 편하다면 `print(row[0], row[-4], row[-3])`처럼 음수 인덱스로 표현해도 됩니다. 실행결과는 같습니다.

이번에는 가맹점명의 데이터 타입을 출력해 봅시다.

```
for row in data:
    print(row[5], type(row[5]))
```

```
실행결과
네이버파이낸셜(주) <class 'str'>
GS샵 <class 'str'>
G마켓 <class 'str'>
...
```

`str`, 즉 문자열로 나옵니다. 이용금액의 데이터 타입도 출력해 봅시다. 이용금액은 숫자니까 당연히 `int`가 나오지 않을까요?

```
for row in data:
    print(row[6], type(row[6]))
```

```
실행결과
546800 <class 'str'>
60440 <class 'str'>
91520 <class 'str'>
...
```

예상과 달리 문자열인 `str`이 나옵니다. `input()`으로 데이터를 입력받으면 문자로 인식했죠. 이와 마찬가지로 파일에서 읽어 들인 데이터도 문자로 인식합니다. 그래서 형태가 숫자라 하더라도 데이터 타입은 모두 문자열로 나옵니다.

앞서 반복문에서 `print(row)`로 이용내역을 한 줄씩 출력했을 때를 잘 살펴보세요. 첫 번째 이용내역인 `['2019-10-12 9:13', '1972753', '본', 'S&', '185', '네이버파이낸셜(주)', '546800', '일시불(A)', '부분취소']`에서도 두 번째 요소인 승인번호 `'1972753'`과 일곱 번째 요소인 이용금액 `'546800'`의 형태는 숫자인데 양옆에 작은따옴표가 있죠. 문자열로 인식됐다는 의미입니다.

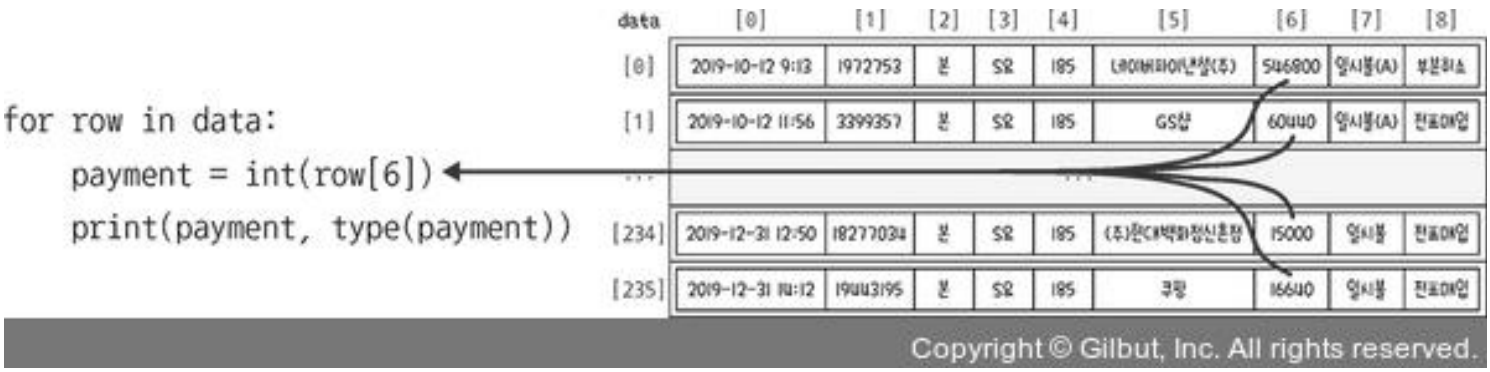
사실 `data` 리스트의 모든 요소가 작은따옴표로 감싸져 있습니다. 즉, CSV 파일에서 읽어 들인 데이터는 모두 문자로 인식됩니다. 따라서 이용금액 등의 데이터를 연산하고 싶으면 문자열을 숫자형으로 바꿔 주는 데이터 타입 변환이 필요합니다. 이용금액은 소수점이 없는 정수이므로 `int()`로 감싸 주면 되겠죠? 변수를 사용해 결괏값을 따로 저장해 봅시다. 이용금액 `row[6]`을 `int()`로 감싸서 변수 `payment`에 저장한 후 출력합니다.

```
for row in data:
    payment = int(row[6])
    print(payment, type(payment))
```

실행결과

```
546800 <class 'int'>
60440 <class 'int'>
91520 <class 'int'>
...
```

이용금액이 정수형으로 바뀐 것을 확인할 수 있습니다. `payment` 변수에는 반복문이 실행될 때마다 다른 값이 들어갑니다. 반복할 때마다 각기 다른 행(일차원 리스트)이 `row` 변수에 들어가고, 달라지는 `row`의 열 인덱스 [6]에 해당하는 값이 `payment` 변수에 저장되기 때문에 그렇습니다. 그림으로 표현하면 다음과 같습니다.



Copyright © Gilbut, Inc. All rights reserved.

반복 회차	row에 담기는 데이터	payment
1번째	data[0] ['2019-10-12 9:13', '1972753', '본', 'S2', '185', '네이버파이낸셜(주)', '546800', '일시불(A)', '부분취소']	546800
2번째	data[1] ['2019-10-12 11:56', '3399357', '본', 'S2', '185', 'GS샵', '60440', '일시불(A)', '전표매입']	60440
...		
235번째	data[234] ['2019-12-31 12:50', '18277034', '본', 'S2', '185', '(주)현대백화점신촌점', '15000', '일시불', '전표매입']	15000
236번째	data[235] ['2019-12-31 14:12', '19443195', '본', 'S2', '185', '쿠팡', '16640', '일시불', '전표매입']	16640

Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 9-8 반복문의 실행과정

가맹점명도 변수에 저장해 봅시다. 변수 `store`에 가맹점명이 들어 있는 `row[5]`를 저장하고 '00(가맹점명)에서 000원(이용금액) 결제' 형식으로 출력해 봅시다.

```
for row in data:
    payment = int(row[6])
    store = row[5]
    print(store, '에서', payment, '원 결제')
```

```
실행결과
네이버파이낸셜(주) 에서 546800 원 결제
GS샵 에서 60440 원 결제
G마켓 에서 91520 원 결제
...
```

모든 이용내역이 의도한 대로 출력됩니다. 이전 코드와 마찬가지로 변수 `store`와 `payment`에 저장되는 값은 반복할 때마다 따라 달라집니다.

이번에는 이용일시도 넣어서 출력해 봅시다. 이용일시는 첫 번째 열에 있습니다. 그런데 '2019-10-12 9:13'처럼 시간도 포함되어 있습니다. 시간은 제외하고 일자만 출력하려면 어떻게 할까요? 힌트는 2019-10-12와 9:13 사이에 있는 공백입니다.

문자열을 분리하는 명령어 `split()`을 사용하면 됩니다. `split()`은 괄호 안에 아무것도 넣지 않으면 자동으로 공백을 기준으로 문자열을 분리합니다. 따라서 공백을 기준으로 `['2019-10-12', '9:13']`으로 나눈 후, 앞부분만 선택하면 됩니다. 이용일시가 들어 있는 `row[0]`을 `split()`으로 분리한 후 `row`의 첫 번째 요소만 출력해 봅시다.

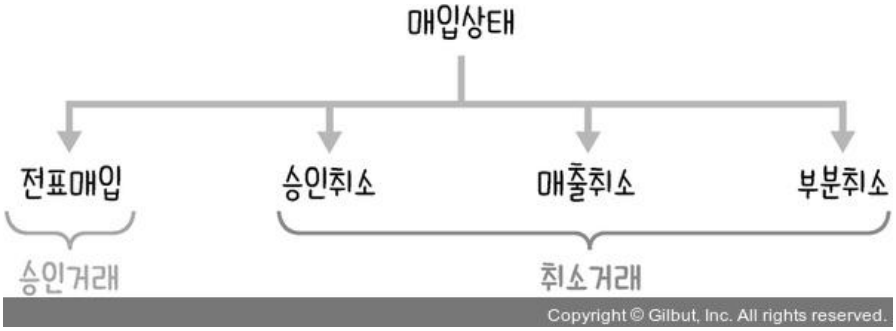
```
for row in data:
    date = row[0].split()[0]
    payment = int(row[6])
    store = row[5]
    print(date, store, '에서', payment, '원 결제')
```

```
실행결과
2019-10-12 네이버파이낸셜(주) 에서 546800 원
2019-10-12 GS샵 에서 60440 원
2019-10-13 G마켓 에서 91520 원 결제
...
```

`row[0].split()[0]`으로 이용일시를 분리해 변수 `date`에 저장하고 이를 출력하니 일자만 출력됩니다. 전체 거래 내역이 들어 있는 리스트 `data`와 날짜를 저장한 변수 `date`는 철자가 비슷하니 입력할 때 유의하세요!

반복문과 조건문으로 데이터 추출하기

이번에는 매입상태에 따라 구매확정 건과 구매취소 건을 구분해 출력해 보겠습니다. 매입상태란 단어가 조금 생소할 텐데요. 매입상태는 카드 이용내역이 승인거래인지 취소거래인지를 나타냅니다. CSV 파일을 잘 살펴보면 크게 전표매입, 승인취소, 매출취소, 부분취소라는 4가지 매입상태가 있습니다.



▲ 그림 9-9 데이터 파일의 매입상태에 따른 구분

'취소'를 포함한 단어와 포함하지 않은 단어로 나눌 수 있죠. 유일하게 '취소'가 들어가지 않은 '전표매입'은 승인거래인데, 승인거래란 카드 사용자의 결제가 성공해서 판매자에게 금액을 지급해야 할 거래입니다. 승인취소와 매출취소는 취소 시점의 차이만 있을 뿐 둘 다 이용금액 결제가 모두 취소된 건입니다. 부분취소는 이용금액 중 일부만 취소된 건을 의미합니다. 원래 표시된 금액만큼 결제했으나 이후에 일부 금액이 취소된 건이죠. 이 데이터로는 부분취소 건의 실제 결제금액을 알 수 없습니다. 그래서 전표매입 건만 '구매확정'으로 출력하고, 승인취소, 매출취소, 부분취소는 모두 '구매취소'로 출력하겠습니다.

모든 이용내역에 대해 '구매확정!' 또는 '구매취소!' 둘 중 하나를 출력해야 합니다. 지금까지 진행한 대로 `data`를 반복문으로 돌며 각 행의 매입상태를 확인하면 됩니다. 만약 매입상태가 '전표매입'이면 '구매확정!'을, '전표매입'이 아니라면 '구매취소!'를 출력합니다. 2가지 상황이 있으므로 조건문이 필요하고요.

```
for row in data:
    if row[-1] == '전표매입':
        print('구매확정!')
    else:
        print('구매취소!')
```

```
실행결과
구매취소!
구매확정!
구매확정!
...
```

매입상태를 나타내는 열이 [-1]이므로 조건문으로 `row[-1]`의 값을 확인합니다. `row[-1]`의 값이 '전표매입'인 경우(`if`)와 그렇지 않은 경우(`else`)로 나누면 되죠. 그리고 2가지 경우에 실행할 출력 명령어를 각각 작성하면 조건에 따라 결과가 출력됩니다.

지금까지 데이터 분석 작업에 들어가기 전에 분석할 데이터를 준비하는 과정을 살펴봤습니다. 이차원 리스트로 저장된 데이터를 이리저리 다뤄 보기도 했고요. 읽어 들인 CSV 파일이 이차원 리스트로 저장되는 과정을 이해하고 있어야 실제 데이터 분석 과정에서 수월하게 진행할 수 있습니다. 만약 이해되지 않는 부분이 있다면 코드를 한 줄씩 살펴보며 복습하기를 권합니다!

NOTE 엑셀 파일을 CSV 파일로 변경하는 방법

분석하고 싶은 데이터가 엑셀 파일일 때 CSV 파일로 간단하게 변경할 수 있습니다. 엑셀 파일을 연 후 [파일 → 다른 이름으로 저장 → 파일 형식을 'CSV (쉼표로 분리)' 선택] 순서로 저장하면 됩니다. 엑셀 버전에 따라 'CSV (쉼표로 분리)' 외에 'CSV UTF-8
(쉼표로 분리)' 형식이 있을 수도 있습니다. 이럴 경우 'CSV (쉼표로 분리)' 대신 'CSV UTF-8 (쉼표로 분리)'를 선택하세요.



▲ 그림 9-10 엑셀 파일을 CSV 파일로 바꾸기

1분 퀴즈

1 다음 코드를 보고 틀린 설명을 고르세요.

```
for row in data:
    payment = int(row[-3])
    month_pay = row[-2]
    print((가), '원', (나), '로 결제')
```

실행결과

```
546800 원 일시불(A) 로 결제
60440 원 일시불(A) 로 결제
...
```

① 첫 번째 줄은 `data` 리스트의 첫 번째 요소부터 마지막 요소까지 반복하는 반복문이다.

② 두 번째 줄은 이용금액을 `payment` 변수에 저장한다.

③ CSV 파일을 읽으면 모든 데이터가 문자로 인식되므로 숫자 연산을 하려면 두 번째 줄처럼 데이터 타입을 바꿔야 한다.

④ (가)에는 `month_pay`, (나)에는 `payment`가 들어가야 한다.

⑤ `row`와 `payment`, `month_pay`에는 반복할 때마다 각각 다른 데이터가 들어간다.

정답 및 해설

9.3

데이터 분석 도전: 카드 이용내역 분석하기

파일로 된 데이터를 읽어 들여 필요한 데이터를 골라내는 방법을 살펴봤습니다. 이번에는 데이터를 가공해 원하는 정보를 뽑아내는 데이터 분석 작업을 어떻게 수행하는지 살펴보겠습니다.

9.3.1 총 지출액 구하기

앞 절에서 읽어 들인 데이터를 활용해 3개월간 카드를 얼마나 사용했는지 총 지출액을 구해 보겠습니다. 코드를 작성하기 전에 어떻게 프로그래밍할지 생각해 봅시다. 총 지출액은 이용금액을 모두 더하면 될 것 같군요. 나중에 활용할 수 있게 총 지출액을 변수에 저장하면 좋겠죠? 총 지출액을 저장하는 변수이므로 `spend`로 합시다. `spend` 변수에 각 행의 이용금액을 더합니다. 이때 파일에서 읽어온 이용금액은 문자로 인식하므로 앞에서 설명했듯이 연산할 수 있는 정수형으로 변환해서 더해야 합니다.

```
spend = int(data[0][-3]) + int(data[1][-3]) + int(data[2][-3]) + …
```

그런데 이용금액이 236개이므로 이를 일일이 더하려면 매우 힘듭니다. 이럴 때 반복문을 사용하면 쉽게 더할 수 있습니다. 반복되는 부분이 보이나요? `int(data[n][-3])` 형태가 계속해서 `spend`에 더해집니다. 이 부분이 반복문 내부에 들어가야 합니다. 여기서 `data[n]`은 이차원 리스트의 행을 나타내므로, 이차원 리스트 `data`의 첫 행부터 마지막 행까지 차례대로 반복되는 반복문이어야 합니다.

그리고 5장에서 반복문으로 숫자 합을 구할 때(5.1.3 반복문으로 간단한 프로그램 만들기 참고) 계산에 사용되는 변수에는 초깃값을 넣어 줘야 한다고 했습니다. 따라서 반복문이 시작되기 전에는 변수 `spend`를 만들면서 초깃값 0을 저장합니다. 이제 반복문으로 전체 카드 이용내역이 담긴 이차원 리스트 `data`의 처음부터 마지막까지 각 행을 `row`에 저장하고 건별 이용금액을 `spend`에 더합니다. 프로그램의 흐름을 정리해 봅시다.

〈프로그램 흐름〉

1. 총 지출액을 저장할 변수(spend)를 만들고 초깃값 0 저장하기
2. 전체 이용내역(data)의 처음부터 마지막까지 반복하기
   2.1 이용금액(data[n][-3])만 골라 정수형으로 변환한 후 총 지출액(spend)에 더하기

그런데 여기서 한 가지 더 고려해야 할 것이 있습니다. 현재 데이터 파일에는 승인거래로 결제가 완료된 건도 있지만, 승인취소, 매입취소, 부분취소 등으로 결제가 취소된 건도 섞여 있습니다. 결제가 취소됐는데 지출액에 포함하면 안 되겠죠. 따라서 한 가지 조건을 추가해야 합니다. 매입상태가 '전표매입'인 건에 한해서만 이용금액을 `spend`에 더해야 합니다(부분취소는 정확히 얼마가 결제됐는지 파일만으로 알기 어렵기 때문에 여기서는 취소거래로 처리합니다). 조건을 추가해 다시 정리해 봅시다.

〈프로그램 흐름〉

1. 총 지출액을 저장할 변수(spend)를 만들고 초깃값 0 저장하기
2. 전체 이용내역(data)의 처음부터 마지막까지 반복하기
   2.1 각 이용내역(data[n][-1])에서 매입상태가 '전표매입'이라면
      2.1.1 이용금액(data[n][-3])만 골라 정수형으로 변환한 후 총 지출액(spend)에 더하기

승인거래임을 확인하는 조건을 과정 2.1에 추가했습니다. 이제 각 과정을 코드로 옮기면 됩니다(지면상 코드 부분 들여쓰기는 생략했습니다).

> 〈프로그램 흐름〉
>
> 1. 총 지출액을 저장할 변수(spend)를 만들고 초깃값 0 저장하기
> 2. 전체 이용내역(data)의 처음부터 마지막까지 반복하기
>   2.1 각 이용내역(data[n][-1])에서 매입상태가 '전표매입'이라면
>   2.1.1 이용금액(data[n][-3])만 골라 정수형으로 변환한 후 총 지출액(spend)에 더하기
>
> ---
>
> 〈코드〉
>
> ```
> 1. spend = 0
> 2. for row in data:
>    2.1 if row[-1] == '전표매입':
>        2.1.1. spend += int(row[-3])
>        // 또는
>        payment = int(row[-3])
>        spend += payment
> ```

매입상태는 열 인덱스 [-1]에 있으므로 과정 2.1에서 조건문 `if row[-1] == '전표매입'`으로 승인거래만 골라낼 수 있습니다. 그리고 조건을 만족할 때만 이용금액을 총 지출액인 `spend` 변수에 더하면 승인거래의 이용금액만 `spend`에 저장됩니다. 이때 `data[n]`이 `row`에 들어가는 일차원 리스트이므로 `data[n][-3]`은 반복문 내부에서는 `row[-3]`과 같습니다. 따라서 합은 `spend += int(row[-3])`으로 구하면 됩니다. 또는 이용금액만을 추출해 `payment` 변수에 저장하고 이 값을 `spend`에 더해도 됩니다. 실행결과는 같습니다. 이는 코딩 스타일의 차이일 뿐이니 선호하는 방식으로 작성하면 됩니다.

정리한 내용을 실제 파이썬 프로그램으로 옮겨 봅시다. 마지막에 `print(spend)`도 넣어 결과를 확인합니다.

```
spend = 0
for row in data:
    if row[-1] == '전표매입':
        payment = int(row[-3])
        spend += payment
print(spend)
```

```
실행결과
8174028
```

3개월간 총 지출액이 8,174,028원이네요. 꽤 많은 돈은 썼군요!

TIP `NameError`가 발생한다면 프로그램을 실행하는 과정에서 CSV 파일을 읽어 들이지 못해서 그렇습니다. 이는 `data` 변수가 만들어지지 않아서 발생하는 오류입니다. CSV 파일은 처음 한 번만 읽어 오면 되므로 앞의 코드 위에 1절에 나온 CSV 파일 읽어 들이기 코드를 추가해야 합니다. 앞으로는 CSV 파일을 읽어 들이는 코드를 이미 실행했다고 가정하고 진행하겠습니다.

이번에는 반복문이 어떻게 진행되는지 살펴봅시다. 반복할 때마다 실행되도록 반복문 내부에 `print(spend)`를 넣습니다.

```
spend = 0
for row in data:
    if row[-1] == '전표매입':
        payment = int(row[-3])
        spend += payment
        print(spend)
```

```
실행결과
0
60440
151960
241770
275370
275370
275370
306970
...
```

`print(spend)`는 조건문 내부에 있기 때문에 매입상태가 '전표매입'일 때만 실행됩니다. 실행결과를 보면 처음에는 0이 나옵니다. 첫 번째 이용내역의 매입상태가 '부분취소'라서 `spend`에 아무것도 더해지지 않았기 때문이죠(데이터 파일을 열고 비교하면서 보세요). 두 번째 반복에서는 매입상태가 '전표매입'이므로 조건문을 실행해 이용금액을 `payment`에 저장한 후 `spend`에 더했습니다. 실행결과에서 `spend` 값이 증가한 것이 보이죠? 세 번째부터 다섯 번째 이용내역도 승인거래여서 `spend`에 이용금액이 더해졌습니다.

그런데 여섯 번째와 일곱 번째 거래는 승인 거래가 아닌가 봅니다. `spend` 값이 275370으로 유지됩니다. CSV 파일을 살펴보면 여섯 번째와 일곱 번째 이용내역의 매입상태가 모두 부분취소입니다. 조건을 충족하지 못해서 이용금액이 `spend`에 더해지지 않았네요.

반복문과 조건문이 어떻게 실행되는지 일곱 번째 반복까지 간단하게 정리해 보겠습니다.

▼ 표 9-1 반복문의 실행과정

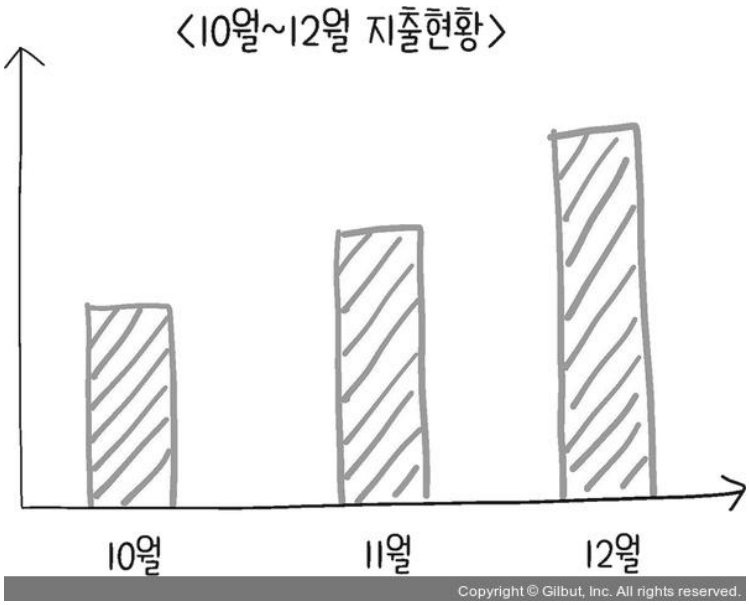
반복 회차	저장된 값			
	row	row[-1]	payment	spend
1	['2019-10-12 9:13', '1972753', '본', 'S&', '185', '네이버파이낸셜(주)', '546800', '일시불(A)', '부분취소']	부분취소	실행되지 않음	0
2	['2019-10-12 11:56', '3399357', '본', 'S&', '185', 'GS샵', '60440', '일시불(A)', '전표매입']	전표매입	60440	60440
3	['2019-10-13 9:16', '12621902', '본', 'S&', '185', 'G마켓', '91520', 'P(A)', '전표매입']	전표매입	91520	151960
4	['2019-10-14 1:32', '20726947', '본', 'S&', '185', '11번가', '89810', '일시불(A)', '전표매입']	전표매입	89810	241770
5	['2019-10-14 7:06', '21234320', '본', 'S&', '185', '네이버파이낸셜(주)', '33600', '일시불(A)', '전표매입']	전표매입	33600	275370
6	['2019-10-15 23:32', '42229559', '본', 'S&', '185', '네이버파이낸셜(주)', '309600', '일시불(A)', '부분취소']	부분취소	33600	275370
7	['2019-10-16 8:00', '43433092', '본', 'S&', '185', 'SSG.COM', '61983', '일시불(A)', '부분취소']	부분취소	33600	275370

여섯 번째 반복을 봅시다. 매입상태가 부분취소이므로 조건문이 실행되지 않아 `payment`에 저장된 값은 다섯 번째 반복에서 저장된 33600이 유지됩니다. `spend`도 마찬가지고요. 일곱 번째 반복도 매입상태가 부분취소이므로 `payment`와 `spend`는 그 이전 값인 33600과 275370을 유지합니다.

지금까지 비교적 간단한 프로그램을 작성했지만, 데이터 분석은 이보다 더 복잡할 때가 많습니다. 무작정 코드를 작성하기보다는 과정을 순서대로 정리하면 더욱 수월하게 코드를 작성할 수 있습니다. 초보자일수록 더욱 그렇고요. 그러니 앞으로 총 지출액 구하는 과정처럼 프로그램 흐름을 순서화하는 연습을 해 보세요!

9.3.2 월별 지출액 그래프 그리기

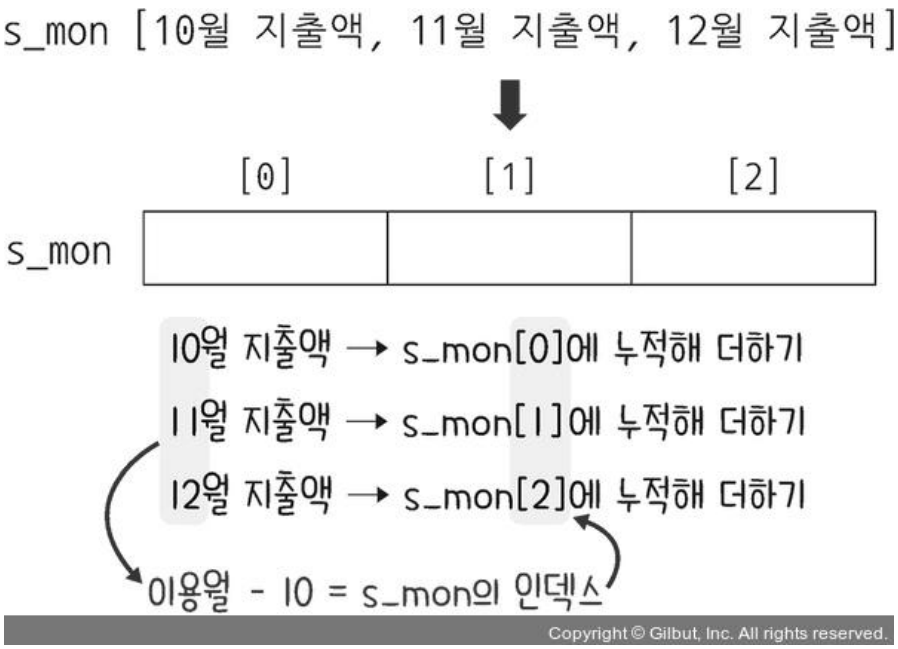
결과를 숫자로만 나열해서는 의미를 파악하기 힘드니 8장에서 배운 내용을 바탕으로 그래프를 그려 결과를 분석해 봅시다. 이번에는 3개월간의 월별 지출액을 구해 그래프를 그립니다. 결과 그래프를 미리 구상해 보면 프로그램의 흐름을 잡는 데 큰 도움이 됩니다. 손으로 직접 그려 보는 것도 좋은 방법입니다. 10~12월 지출현황을 다음 그림처럼 구상해 봅시다.



▲ 그림 9-11 예상 결과 그래프

x축은 월, y축은 월별 지출액을 나타내는 막대그래프로 그립니다. 먼저 막대그래프를 그리려면 `bar()` 명령어에 x축 데이터와 y축 데이터를 리스트 형태로 넣어야 합니다. 따라서 x축에는 `['10월', '11월', '12월']`, y축에는 `[10월 지출액, 11월 지출액, 12월 지출액]`이 들어갑니다.

y축에 들어갈 월별 지출액 리스트는 spend_month를 줄인 `s_mon` 변수에 저장하겠습니다. 그럼 10~12월 지출액은 각각 `s_mon[0]`, `s_mon[1]`, `s_mon[2]`에 들어갑니다. 카드 이용월과 `s_mon` 인덱스 사이의 규칙이 보이나요? (카드 이용월 - 10)을 하면 리스트 `s_mon`의 인덱스가 됩니다. 따라서 `data`에 저장된 이용내역 건마다 이용월에서 10을 빼서 `s_mon`의 인덱스를 구합니다. 그리고 해당 인덱스의 값에 이용금액을 누적해서 더하면 됩니다.



▲ 그림 9-12 카드 이용월과 인덱스의 관계

그럼 월별 지출액은 어떻게 구할까요? 먼저 첫 번째 열의 이용일시에서 월에 해당하는 부분을 골라냅니다. 이용일시는 2019-10-12처럼 연-월-일 사이에 하이픈(-)이 있으므로 하이픈을 기준으로 `split()` 명령어를 사용해 이용일시를 분리할 수 있습니다.

이를 바탕으로 월별 지출액을 구하는 과정을 정리해 보겠습니다. 월별 지출액을 구할 때도 역시 매입상태가 전표매입인 경우(승인거래)에만 지출액에 더해야죠? 따라서 3개월간의 총 지출액을 구하는 과정에 이용월과 `s_mon`의 인덱스 구하는 부분을 추가하면 됩니다. 프로그램 흐름을 종합하면 다음과 같습니다.

〈프로그램 흐름〉

1. 10~12월의 월별 지출액을 저장할 리스트(s_mon)를 만들고 초깃값 0 저장하기
2. 전체 이용내역(data)의 처음부터 마지막까지 반복하기
    2.1 각 이용내역에서 매입상태가 '전표매입'이라면
        2.1.1 해당 건의 이용월과 이용금액 구하기
        2.1.2 이용월에서 10을 빼 월별 지출액 리스트(s_mon)의 인덱스(idx) 구하기
        2.1.3 과정 2.1.2에서 구한 인덱스의 값(s_mon[idx])에 과정 2.1.1에서 구한 이용금액 더하기
3. 월별 지출액 리스트(s_mon)로 막대그래프 그리기

총 지출액을 구할 때는 변수 하나에 이용금액을 계속 더했지만, 월별 지출액을 구할 때는 리스트를 사용합니다. 그래서 이용월에서 10을 빼면 리스트의 인덱스가 된다는 규칙을 찾아 과정 2.1.2과 2.1.3을 진행합니다.

그럼 프로그램 흐름을 코드로 옮겨 보겠습니다. 이용월을 저장하는 변수는 `mon`, 인덱스를 저장하는 변수는 `idx`이고, 나머지 변수는 총 지출액을 구할 때와 동일합니다.

> 〈프로그램 흐름〉
>
> 1. 10~12월의 월별 지출액을 저장할 리스트(s_mon)를 만들고 초깃값 0 저장하기
> 2. 전체 이용내역(data)의 처음부터 마지막까지 반복하기
> 2.1 각 이용내역에서 매입상태가 '전표매입'이라면
> 2.1.1 해당 건의 이용월과 이용금액 구하기
> 2.1.2 이용월에서 10을 빼 월별 지출액 리스트(s_mon)의 인덱스(idx) 구하기
> 2.1.3 과정 2.1.2에서 구한 인덱스의 값(s_mon[idx])에 과정 2.1.1에서 구한 이용금액 더하기
> 3. 월별 지출액 리스트(s_mon)로 막대그래프 그리기
>
> ---
>
> 〈코드〉
>
> ```
> 1. s_mon = [0, 0, 0]
> 2. for row in data:
>  2.1 if row[-1] == '전표매입':
>  2.1.1 mon, payment = row[0].split('-')[1], int(row[-3])
>  2.1.2 idx = mon - 10
>  2.1.3 s_mon[idx] += payment
> 3. plt.bar(['10월', '11월', '12월'], s_mon)
> ```

과정 3에는 8장에서 배운 `matplotlib.pyplot` 라이브러리와 한글 글꼴을 지정하는 등의 코드가 더 있지만, 여기서는 그래프를 그리는 핵심 코드만 표현했습니다. 그럼 과정 2까지 코드로 작성해 봅시다. 마지막에 결과 출력을 위한 코드도 한 줄 추가합니다.

```
s_mon = [0, 0, 0]
for row in data:
    if row[-1] == '전표매입':
        mon, payment = row[0].split('-')[1], int(row[-3])
        idx = mon - 10
        s_mon[idx] += payment
print(s_mon)
```

```
실행결과
----> 5 idx = mon - 10
TypeError: unsupported operand type(s) for -: 'str' and 'int'
```

데이터 타입이 맞지 않을 때 나오는 `TypeError`가 발생합니다. `TypeError`가 발생한 위치는 과정 2.1.2에서 인덱스를 구하는 부분입니다. 오류를 보면 "-(뺄셈) 연산에 지원되지 않는 연산자 타입: '문자열'과 '정수형'"이라고 나옵니다. 뺄셈 연산을 하는데 하나는 문자열이고 하나는 정수형이라서 연산할 수 없다는 뜻이죠. 뒤에 있는 10은 정수가 분명하니 `mon`이 문자겠네요. `mon`에 데이터를 저장한 바로 윗줄을 봅시다. CSV 파일에서 읽어 들인 `data`는 모두 문자열이라고 했습니다. 그런데 `row[0].split('-')[1]`을 보면 이용일시를 분리만 하고 이용월을 정수형으로 바꾸는 작업은 하지 않았습니다. 그래서 아랫줄의 뺄셈 연산에서 오류가 발생한 겁니다.

`row[0].split('-')[1]`을 `int()`로 감싸서 정수형으로 변환하고 다시 실행해 봅시다.

```
s_mon = [0, 0, 0]
for row in data:
    if row[-1] == '전표매입':
        mon, payment = int(row[0].split('-')[1]), int(row[-3])
        idx = mon - 10
        s_mon[idx] += payment
print(s_mon)
```

```
실행결과
[2093015, 4127744, 1953269]
```

이번에는 결과가 제대로 출력됩니다. 코드 실행 과정을 보고 싶다면 반복문에 `print(row[0].split('-'), mon, s_mon)`나 `print(mon, idx, s_mon)` 등의 코드를 넣어 반복할 때마다 달라지는 변수의 값을 직접 확인해 보세요.

> TIP 가끔 '2019-10-12 9:13'이 아니라 '2019/10/12 9:13'나 '2019.10.12 9:13'처럼 하이픈 대신 다른 기호로 날짜가 표시되는 경우가 있습니다. 이럴 때는 반복문에 `print(row[0])`을 넣어 이용일시를 직접 출력해 보고 화면에 보이는 기호를 `split()`에 넣어 주면 됩니다.

이 코드 역시 변수를 사용하지 않고도 작성할 수 있습니다. 변수를 사용하지 않으면 다소 복잡하지만, 놀랍게도 코드가 세 줄에서 한 줄로 줄어듭니다.

```
s_mon = [0, 0, 0]
for row in data:
    if row[-1] == '전표매입':
        s_mon[int(row[0].split('-')[1]) - 10] += int(row[-3])
print(s_mon)
```

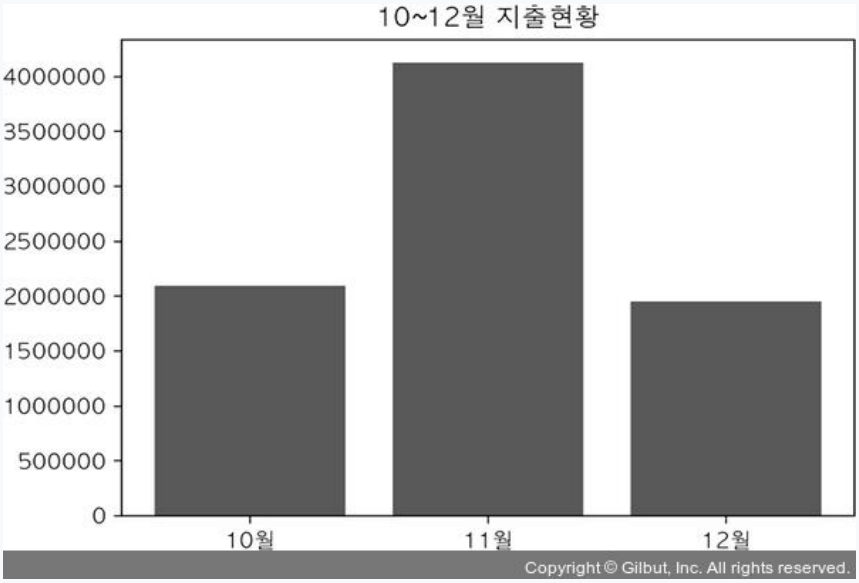
앞의 코드에 이어 과정 3의 그래프를 그려 봅시다.

```
import matplotlib.pyplot as plt

s_mon = [0, 0, 0]
for row in data:
    if row[-1] == '전표매입':
        mon, payment = int(row[0].split('-')[1]), int(row[-3])
        idx = mon - 10
        s_mon[idx] += payment

plt.rc('font', family='Malgun Gothic')
plt.title('10~12월 지출현황')
plt.bar(['10월', '11월', '12월'], s_mon, color='royalblue')
plt.show()
```

실 행 결 과



라이브러리를 포함하는 `import matplotlib.pyplot as plt`를 기존 코드 위에 추가하고, 그래프를 그리는 코드를 기존 코드 아래에 추가했습니다.

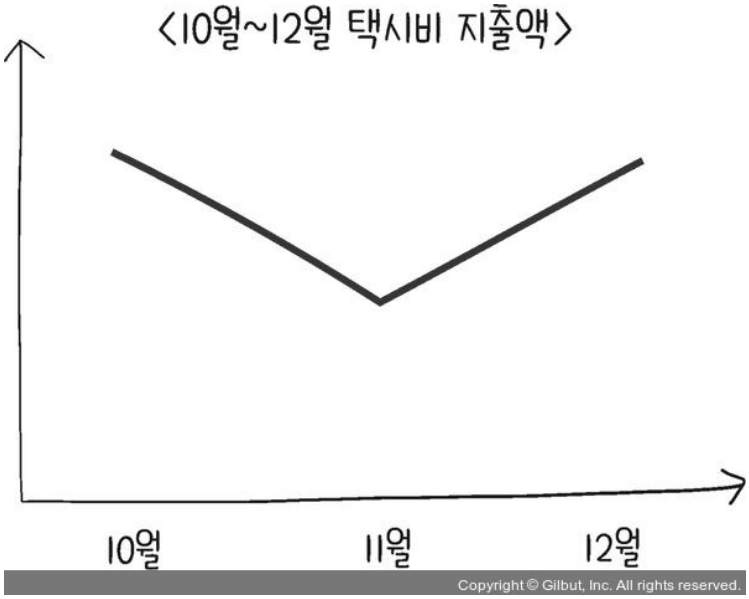
결과 그래프를 보면 10월과 12월은 지출액이 비슷한데, 11월에는 지출액이 다른 달의 두 배 정도 되네요. 11월에 다른 달보다 더 지출한 항목을 찾으면 그 이유를 알 수 있습니다. 데이터 시각화를 계속 진행하면서 그 이유를 찾아봅시다.

9.3.3 지출액 비교 그래프 그리기

이번에는 불필요한 지출이 얼마나 있는지 파악해 봅시다. 여기에서는 택시비와 배달음식비를 불필요한 지출로 보고, 월별 택시비 지출액과 배달음식비 지출액을 비교 그래프로 그려 보겠습니다. 이용내역을 쭉 훑어보면 유독 택시와 배달음식이 자주 등장하는 것이 보이는데요. 실제로 택시와 배달음식에 얼마를 지출했는지 알아봅시다.

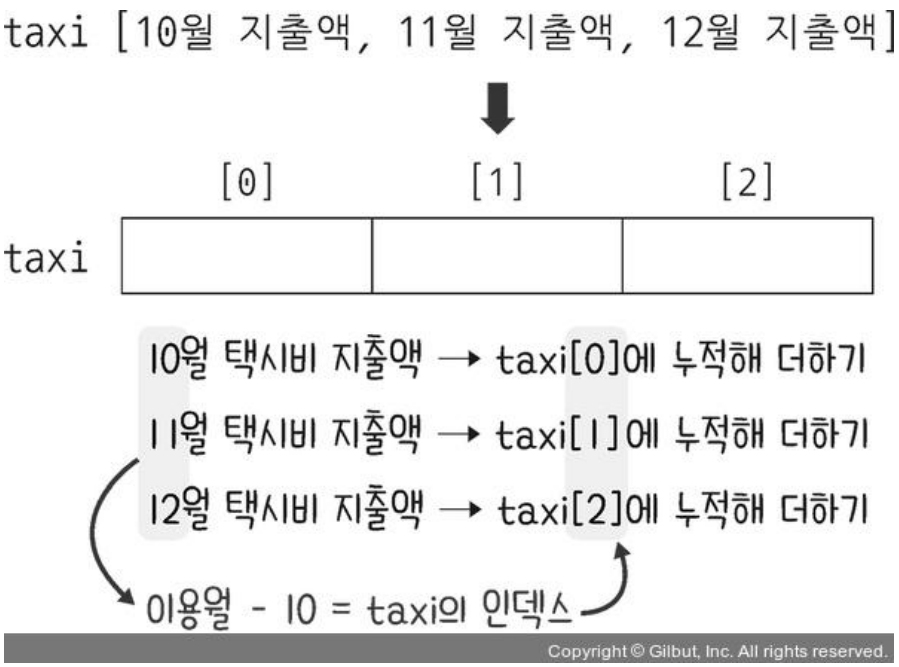
월별 택시비 지출액 그래프 그리기

먼저 택시비 지출액 그래프를 그려 봅시다. 프로그래밍을 시작하기 전에 본인이 원하는 그래프를 간단히 그려 보면 코딩 과정이 편합니다. 이번에는 월별 택시비 지출액을 꺾은선 그래프로 그려 봅시다.



▲ 그림 9-13 예상 결과 그래프

꺾은선 그래프를 그리려면 앞서 월별 지출액을 구했을 때와 마찬가지로 리스트가 필요합니다. `plt.plot()` 명령어에 넣을 x축 데이터는 `['10월', '11월', '12월']`로 같고, y축 데이터는 월별 택시비 지출액을 저장한 리스트(`['10월 택시비 지출액', '11월 택시비 지출액', '12월 택시비 지출액']`)가 들어가면 됩니다. 월별 택시비 지출액을 저장한 리스트를 `taxi`로 이름 짓고 그림으로 나타내 봅시다.



▲ 그림 9-14 택시비 지출액 리스트

월별 지출액처럼 이용월과 `taxi` 리스트의 인덱스는 10 차이가 납니다.

다음으로 전체 이용내역에서 택시 이용 건만 골라내야 합니다. 어떻게 골라낼까요? 파일을 한번 훑어보세요. 정확한 가맹점명을 모르더라도 일단 '택시'라는 키워드가 포함되면 택시 이용내역이겠죠? 따라서 가맹점명에 '택시'가 들어간 건의 이용월과 이용금액을 구하면 됩니다. 월별 지출액을 구한 과정에 '택시'라는 조건을 하나 더 추가하면 되겠네요.

〈프로그램 흐름〉

1. 10~12월의 월별 택시비 지출액을 저장할 리스트(taxi)를 만들고 초깃값 0 저장하기
2. 전체 이용내역(data)의 처음부터 마지막까지 반복하기
   2.1 각 이용내역에서 매입상태가 '전표매입'이고, 가맹점명에 '택시'가 들어 있다면
      2.1.1 해당 건의 이용월과 이용금액 구하기
      2.1.2 이용월에서 10을 빼 월별 택시비 지출액 리스트(taxi)의 인덱스(idx) 구하기
      2.1.3 과정 2.1.2에서 구한 인덱스의 값(taxi[idx])에 과정 2.1.1에서 구한 이용금액 더하기
3. 월별 택시비 지출액 리스트(taxi)로 꺾은선 그래프 그리기

추가 조건인 '택시' 이용내역은 과정 2.1에 넣으면 됩니다. 그러면 자동으로 택시 이용건만 골라낼 수 있습니다. 따라서 과정 2.1의 조건이 2개가 됩니다. 두 조건을 모두 충족해야 하므로 `and`로 연결합니다. 그럼 각 단계를 코드로 옮겨 보죠.

〈프로그램 흐름〉

1. 10~12월의 월별 택시비 지출액을 저장할 리스트(taxi)를 만들고 초깃값 0 저장하기
2. 전체 이용내역(data)의 처음부터 마지막까지 반복하기
   2.1 각 이용내역에서 매입상태가 '전표매입'이고, 가맹점명에 '택시'가 들어 있다면
      2.1.1 해당 건의 이용월과 이용금액 구하기
      2.1.2 이용월에서 10을 빼 월별 택시비 지출액 리스트(taxi)의 인덱스(idx) 구하기
      2.1.3 과정 2.1.2에서 구한 인덱스의 값(taxi[idx])에 과정 2.1.1에서 구한 이용금액 더하기
3. 월별 택시비 지출액 리스트(taxi)로 꺾은선 그래프 그리기

〈코드〉

```
1. taxi = [0, 0, 0]
2. for row in data:
    2.1. if row[-1] == '전표매입' and '택시' in row[5]:
        2.1.1 mon, payment = int(row[0].split('-')[1]), int(row[-3])
        2.1.2 idx = mon - 10
        2.1.3 taxi[idx] += payment
3. plt.plot(['10월', '11월', '12월'], taxi)
```

가맹점명은 열 인덱스 [5]에 있으므로 가맹점명에 '택시'가 포함된 건만 골라내려면 조건 `'택시' in row[5]`를 추가합니다. 그리고 과정 2.1에서 두 조건을 `and`로 연결합니다. 그럼 과정 2까지 프로그램으로 옮겨 봅시다.

```
taxi = [0, 0, 0]
for row in data:
    if row[-1] == '전표매입' and '택시' in row[5]:
        mon, payment = int(row[0].split('-')[1]), int(row[-3])
        idx = mon - 10
        taxi[idx] += payment
print(taxi)
```

실행결과

```
[8600, 83900, 52800]
```

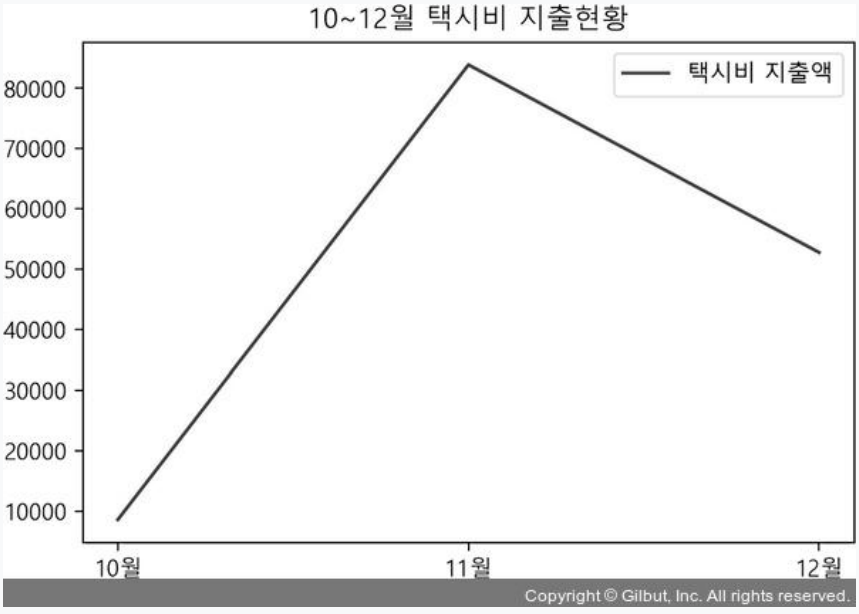
택시비로 지출한 금액들이 `taxi` 리스트에 월별로 저장됐습니다. 월별 지출액을 구하는 코드에서 리스트명을 수정하고, 조건문에 조건 하나만 추가했는데 구해졌습니다. 이렇게 파이썬에서는 간단한 조작만으로도 수많은 데이터에서 원하는 결과를 빠르게 도출할 수 있습니다.

TIP 실행 과정을 확인하고 싶으면 반복문 안에 `print(row[5], mon, taxi)`를 추가해 반복 회차별 변수 값의 변화를 살펴보세요.

그래프를 그릴 때 필요한 택시비 지출액 리스트를 만들었으니 마지막 과정 3의 꺾은선 그래프를 그려 봅시다. 이번에는 그래프에 범례를 표시하고 색상은 `crimson`(진분홍)으로 하겠습니다.

```
plt.rc('font', family='Malgun Gothic')
plt.title('10~12월 택시비 지출현황')
plt. plot(['10월', '11월', '12월'], taxi, color='crimson', label='택시비 지출액')
plt.legend()
plt.show()
```

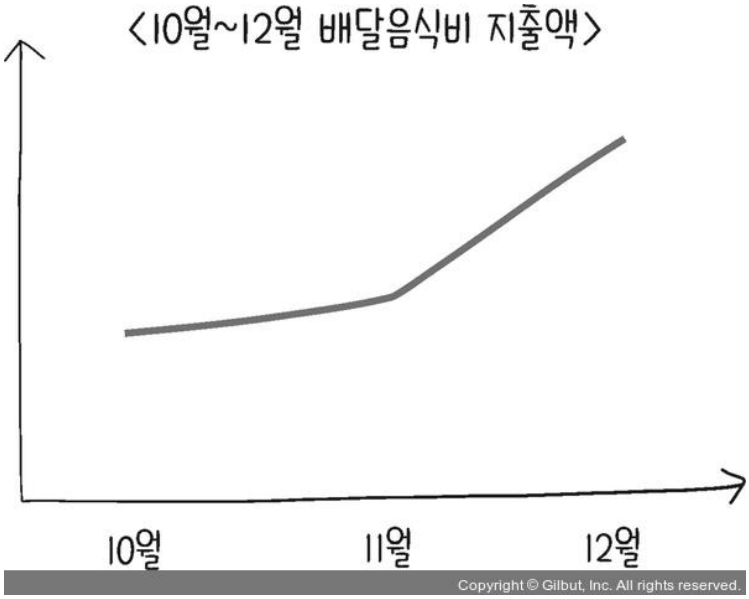
실행결과



꺾은선 그래프는 `plt.plot()`으로 그립니다. `plt.plot()`에 `label` 옵션으로 라벨을 추가하고 `plt.legend()`으로 범례를 표시합니다. 택시비 지출액 역시 11월에 가장 높게 나타나네요.

월별 배달음식비 지출액 그래프 그리기

다음으로 월별 배달음식비 지출액을 구합니다. 배달음식비 지출액 그래프도 미리 구상해 봅시다. 배달음식비 지출액 그래프도 택시비 지출액 그래프와 비교할 수 있게 꺾은선 그래프로 표현합니다.



▲ 그림 9-15 예상 결과 그래프

앞에서 작성한 택시비 지출액 그래프와 비슷합니다. 배달음식비 지출액 역시 리스트가 필요하고, 리스트에 배달음식 업체에서 사용한 금액만 누적하면 됩니다. 따라서 가맹점명을 택시에서 배달음식점으로 변경해야죠.

카드 이용내역 파일을 다시 봅시다. 배달음식을 자주 이용한다면 단번에 알 수 있습니다. 배달 주문 서비스 업체인 '(주)우아한형제들'이 보이나요? 이곳에서 이용한 내역만 뽑아서 배달음식비 지출액을 구합니다. 가맹점명이 '(주)우아한형제들'인 이용 건에 대해 리스트의 인덱스를 구한 후, 이용금액을 모두 더하면 되겠죠? 월별 배달음식비 지출액을 저장하는 리스트는 delivery의 약자 `deli`로 하겠습니다. 기존 코드와 비슷하니 이번에는 프로그램 흐름과 코드를 한 번에 정리해 봅시다.

〈프로그램 흐름〉

1. 10~12월의 월별 배달음식비 지출액을 저장할 리스트(deli)를 만들고 초깃값 0 저장하기
2. 전체 이용내역(data)의 처음부터 마지막까지 반복하기
   2.1 각 이용내역에서 매입상태가 '전표매입'이고, 가맹점명이 '(주)우아한형제들'이라면
      2.1.1 해당 건의 이용월과 이용금액 구하기
      2.1.2 이용월에서 10을 뺀 월별 배달음식비 지출액 리스트(deli)의 인덱스(idx) 구하기
      2.1.3 과정 2.1.2에서 구한 인덱스의 값(deli[idx])에 과정 2.1.1에서 구한 이용금액 더하기
3. 월별 배달음식비 지출액 리스트(deli)로 꺾은선 그래프 그리기

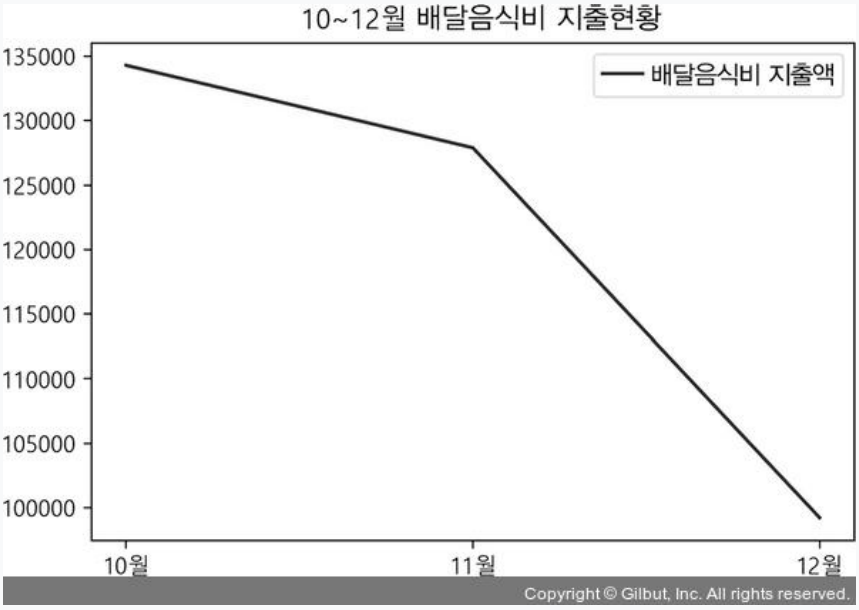
〈코드〉

```
1. deli = [0, 0, 0]
2. for row in data:
    2.1 if row[-1] == '전표매입' and row[5] == '(주)우아한형제들':
        2.1.1 mon, payment = int(row[0].split('-')[1]), int(row[-3])
        2.1.2 idx = mon - 10
        2.1.3 deli[idx] += payment
3. plt.plot(['10월', '11월', '12월'], deli)
```

과정 2.1에서 조건을 택시 대신 (주)우아한형제들로 변경하면 됩니다. 코드를 프로그램으로 옮기고 그래프도 바로 그려 봅시다. 이번에는 꺾은선 그래프를 `indigo`(남색)로 그려 보겠습니다.

```
deli = [0, 0, 0]
for row in data:
    if row[-1] == '전표매입' and row[5] == '(주)우아한형제들':
        mon, payment = int(row[0].split('-')[1]), int(row[-3])
        idx = mon - 10
        deli[idx] += payment
plt.rc('font', family='Malgun Gothic')
plt.title('10~12월 배달음식비 지출현황')
plt. plot(['10월', '11월', '12월'], deli, color='indigo', label='배달음식비 지출액')
plt.legend()
plt.show()
```

실행결과



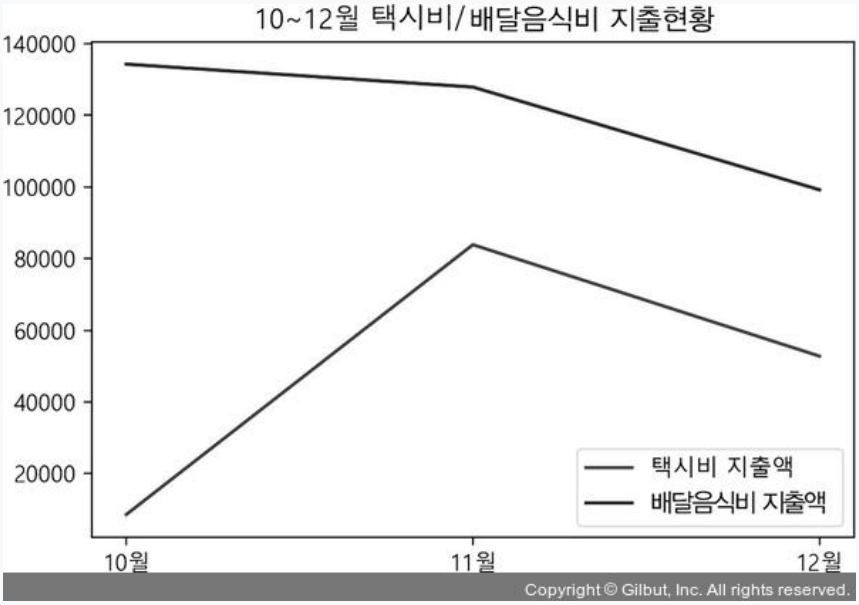
배달음식비 지출액 그래프도 잘 그려집니다.

비교 그래프 그리기

마지막으로 두 그래프를 합쳐서 비교 그래프를 그려 보겠습니다. 2개의 꺾은선 그래프를 합치려면 2개의 `plt.plot()` 명령어 후에 `plt.show()` 하나를 사용하면 됩니다.

```
plt.title('10~12월 택시비/배달음식비 지출현황')
plt. plot(['10월', '11월', '12월'], taxi, color='crimson', label='택시비 지출액')
plt. plot(['10월', '11월', '12월'], deli, color='indigo', label='배달음식비 지출액')
plt.legend()
plt.show()
```

실행결과



2개의 꺾은선 그래프가 그려집니다. 택시보다 배달음식에 돈을 더 많이 썼네요. 10월에는 택시비 지출액이 가장 적었지만, 배달음식비 지출액은 가장 많았어요. 외부 활동보다는 집에서 보내는 시간이 많았을 것으로 추측할 수 있습니다. 11월은 택시비와 배달음식비 둘 다 지출액이 많네요. 12월에는 택시비와 배달음식비 지출액 모두 전 달보다 조금 덜 썼습니다.

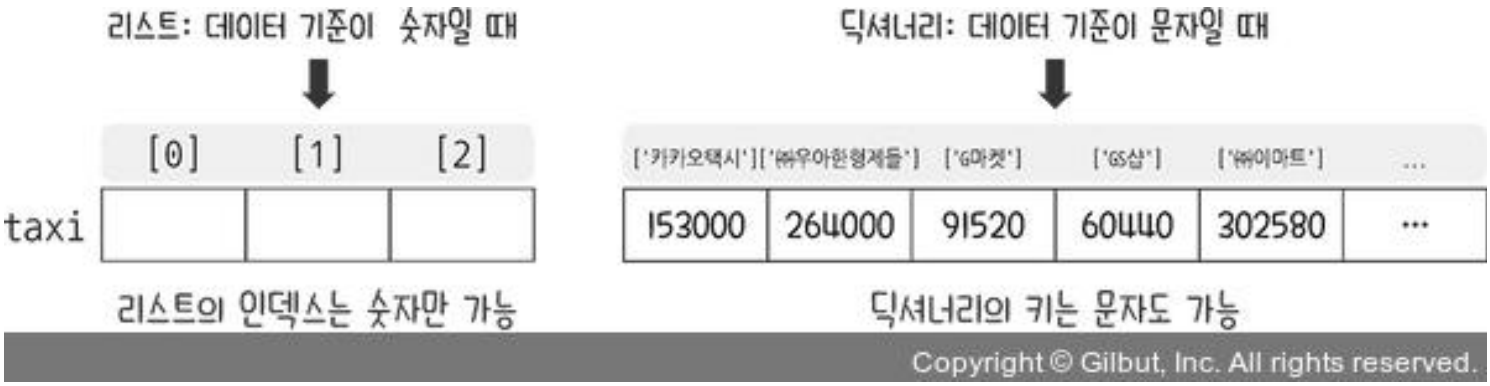
지금까지 CSV 파일을 이용해 간단한 데이터 분석을 해 보았습니다. 이렇게 코드 몇 줄로 월별 지출액 추이를 한눈에 볼 수 있도록 시각화할 수 있습니다. 응용한다면 더 다양한 재미있는 그래프를 그릴 수 있습니다. 직접 응용해서 그래프를 그려 보세요.

9.3.4 지출액 상위 10개 항목 뽑기

이번에는 3개월간 어디에 가장 많은 돈을 썼는지 알아봅시다. 먼저 가맹점별 지출액을 알아야죠. A가맹점 - 000원, B가맹점 - 000원, C가맹점 - 000원 등으로 가맹점별 지출액을 구한 후, 지출액을 기준으로 내림차순 정렬하면 지출액이 큰 내역부터 작은 내역순으로 정렬됩니다.

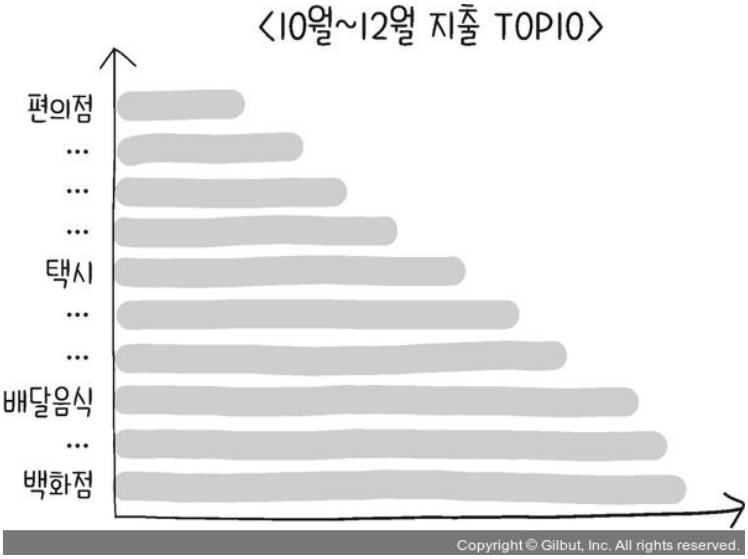
지금까지 월별 지출액은 리스트로 구했습니다. 월을 기준으로 이용금액을 분류하기 때문에 숫자 인덱스를 사용하는 리스트가 유리합니다. 그런데 이번에는 가맹점을 기준으로 이용금액을 분류해야 합니다. 즉, 데이터 기준이 숫자가 아닌 문자입니다. 이런 경우에는 문자를 키로 쓰는 딕셔너리를 사용합니다.

다음 그림처럼 딕셔너리를 사용해 가맹점명을 키로, 해당 가맹점에서 사용한 금액을 값으로 하면 됩니다.



▲ 그림 9-16 리스트와 딕셔너리 비교

딕셔너리를 활용해 돈을 많이 쓴 가맹점 10개를 골라 다음 그림처럼 가로형 막대그래프로 그려 봅시다.



▲ 그림 9-17 예상 결과 그래프

프로그램을 어떻게 만들지 생각해 봅시다. 먼저 가맹점별 지출액을 저장하는 딕셔너리를 만들어야죠. 딕셔너리의 이름은 `spending`으로 합시다. `spending`은 처음에는 빈 딕셔너리입니다. 카드 이용내역을 처음부터 마지막까지 확인하면서 매입 상태가 전표매입인 건을 골라냅니다. 여기까지는 익숙하죠?

다음으로 전표매입 건에서 가맹점명과 이용금액을 구하고, `spending` 딕셔너리에서 해당 가맹점명을 키로 하는 값에 이용금액을 더하면 됩니다. 이때 2가지 경우가 있습니다. 해당 가맹점명이 키에 있는 경우와 없는 경우입니다. 만약 `spending`에 해당 가맹점명을 키로 하는 요소가 없다면 새로 키를 추가하고, 해당 가맹점명을 키로 하는 요소가 있다면 해당 요소의 값에 이용금액을 더합니다. 전체 이용내역에서 이 과정을 완료했다면 지출액을 기준으로 딕셔너리를 내림차순 정렬합니다. 그리고 앞에서 10개 요소를 선택하면 이것이 지출액 상위 10개 항목입니다.

그래프를 그릴 재료는 모두 준비했습니다. 다만, 그래프를 그릴 수 있도록 딕셔너리를 살짝 변경해야 합니다. 가로형 막대그래프를 그리려면 `plt.barh()`를 사용해야죠. `plt.barh()`에는 리스트 2개가 들어갑니다. 그림 9-17을 보면 상위 10개 가맹점을 담은 리스트와 각 가맹점의 지출액을 담은 리스트가 필요합니다. 그런데 이 두 리스트는 이미 앞에서 구한 `spending` 딕셔너리의 키와 값입니다. 따라서 딕셔너리의 키와 값을 다음과 같이 각각 리스트로 만드는 과정이 필요합니다.



▲ 그림 9-18 딕셔너리의 키와 값을 각각 리스트로 만들기

프로그램 흐름을 정리해 봅시다.

〈프로그램 흐름〉

1. 가맹점별 지출액을 저장할 빈 딕셔너리(spending) 만들기
2. 전체 이용내역(data)의 처음부터 마지막까지 반복하기
    2.1 각 이용내역에서 매입상태가 '전표매입'이라면
        2.1.1 해당 건의 가맹점명(store)과 이용금액(payment) 구하기
        2.1.2 만약 딕셔너리의 키에 해당 가맹점명(store)이 없다면
        2.1.2.1 가맹점명(store)을 키로, 이용금액(payment)을 값으로 하는 요소 추가하기
        2.1.3 만약 딕셔너리의 키에 해당 가맹점명(store)이 있다면
        2.1.3.1 해당 요소에 이용금액(payment) 더하기
3. 값(payment)을 기준으로 완성된 딕셔너리(spending)를 내림차순 정렬한 다음, 앞에서 10개 요소 뽑기
4. 지출액 상위 10개의 가맹점명과 지출액(spending의 키와 값)을 각각 리스트에 저장하기
5. 가맹점명 리스트와 지출액 리스트로 막대그래프 그리기

과정 2와 3이 프로그램의 핵심입니다. 과정 2에서 가맹점별 지출액을 모두 구하고, 과정 3에서 지출액 상위 10개 항목을 골라냅니다. 그리고 과정 4에서 `plt.bar()`에 넣을 리스트를 만듭니다.

> 제제쌤의 조언
> 과정 3까지 이해되지 않는다면 7장에서 영어 단어장 만들기 과정을 참고하기 바랍니다.

단계가 많고 조금 복잡하니 나눠서 진행해 보겠습니다.

가맹점별 지출액을 저장하는 딕셔너리 만들기
우선 과정 2.1까지만 코드를 작성해 봅시다.

> 〈프로그램 흐름〉
>
> 1. 가맹점별 지출액을 저장할 빈 딕셔너리(spending) 만들기
> 2. 전체 이용내역(data)의 처음부터 마지막까지 반복하기
> 2.1 각 이용내역에서 매입상태가 '전표매입'이라면
> 2.1.1 해당 건의 가맹점명(store)과 이용금액(payment) 구하기
> 2.1.2 만약 딕셔너리의 키에 해당 가맹점명(store)이 없다면
> 2.1.2.1 가맹점명(store)을 키로, 이용금액(payment)을 값으로 하는 요소 추가하기
> 2.1.3 만약 딕셔너리의 키에 해당 가맹점명(store)이 있다면
> 2.1.3.1 해당 요소에 이용금액(payment) 더하기
>
> ---
>
> 〈코드〉

```
1. spending = {}
2. for row in data:
    2.1 if row[-1] == '전표매입':
        2.1.1 store, payment = row[-4], int(row[-3])
        2.1.2 if store not in spending.keys():
            2.1.2.1 spending[store] = payment
        2.1.3 else:
            2.1.3.1 spending[store] += payment
```

과정 2.1.1까지는 익숙한 코드죠. 과정 2.1.2와 2.1.3은 `if-else` 문으로 표현할 수 있습니다. `spending` 딕셔너리의 키에 가맹점명 `store`가 없다면 {키:값}이 `{store:payment}`인 요소를 추가하고, 키에 `store`가 있다면 해당 요소에 이용금액 `payment`를 더합니다. 모든 이용내역에 대해 이 과정을 반복하면 가맹점명과 이용금액을 키-값으로 하는 `spending` 딕셔너리를 완성할 수 있습니다. 여기까지 작성해 봅시다.

```
spending = {}
for row in data:
    if row[-1] == '전표매입':
        store, payment = row[-4], int(row[-3])
        if store not in spending.keys():
            spending[store] = payment
        else:
            spending[store] += payment
print(spending)
```

`실행결과`

```
{'GS샵': 60440, 'G마켓': 201630, '11번가': 151010, '네이버파이낸셜(주)': 1395840, '(주)에이랜드타임스퀘어': 31600, '(주)곱': 96000, '카카오페이(택시)': 78000, '포도당': 75000, …}
```

가맹점명과 해당 가맹점에서 지출한 금액이 쌍을 이룬 긴 딕셔너리가 출력됩니다.

이번에는 `print(spending)`을 반복문 내부에 넣어 반복 회차별로 딕셔너리가 어떻게 변하는지 확인해 봅시다.

> 실행결과
> ```
> {}
> {'GS샵': 60440}
> {'GS샵': 60440, 'G마켓': 91520}
> {'GS샵': 60440, 'G마켓': 91520, '11번가': 89810}
> {'GS샵': 60440, 'G마켓': 91520, '11번가': 89810, '네이버파이낸셜(주)': 33600}
> ...
> ```

반복할 때마다 새 요소가 추가됩니다. 첫 번째 `if` 문을 충족한 첫 번째 승인거래의 키는 'GS샵'입니다. 딕셔너리의 키에 GS샵이 없기 때문에 두 번째 `if` 문 또한 충족해 `spending[store] = payment`를 실행합니다. 따라서 GS샵을 키로, 이용금액 60440을 값으로 하는 요소가 추가됩니다. 두 번째 승인거래도 G마켓을 키로 하는 요소가 없으므로 `spending` 딕셔너리에 추가됩니다. 이렇게 반복문을 완료하면 `spending` 딕셔너리가 완성됩니다.

지출액 상위 10개 가맹점 골라 그래프 그리기
다음으로 과정 3과 4를 코드로 작성해 봅시다.

> 〈프로그램 흐름〉
>
> 3. 값(payment)을 기준으로 완성된 딕셔너리(spending)를 내림차순 정렬한 다음, 앞에서 10개 요소 뽑기
>
> ---
>
> 〈코드〉
>
> ```
> 3. top10 = sorted(spending.items(), key=operator.itemgetter(1), reverse=True)[:10]
> ```

과정 3에서는 값을 기준으로 `spending` 딕셔너리를 내림차순 정렬해야 하므로 `operator` 라이브러리의 `itemgetter()`와 `reverse=True` 옵션을 활용합니다(7장 참고). 이렇게 만들어진 리스트에서 앞의 10개 항목을 추출해야 하므로 `[:10]`으로 슬라이싱한 결과를 `top10`에 저장합니다.

과정 3까지 작성하고 `top10`을 출력하면 리스트 요소가 소괄호로 감싸진 형태, `[('네이버파이낸셜(주)', 1395840), ('스마일페이 (신라면세점)', 978035)...]`로 출력됩니다. 이처럼 소괄호로 감싸진 한 단위를 튜플(tuple)이라고 하는데. 이 책에서는 자세히 다루지 않았습니다.

과정 4를 진행하려면 튜플 안에 있는 가맹점명과 이용금액을 분리해야 합니다. 여기서 바로 반복문으로 튜플의 요소를 분리하는 방법을 배워 보겠습니다. 간단합니다. 튜플의 요소에는 리스트처럼 인덱스로 접근할 수 있습니다. 따라서 `for t in top10` 으로 반복문을 작성하면 `top10`을 구성하는 튜플이 하나씩 `t`에 저장됩니다. 튜플의 요소에 접근하려면 인덱스를 활용해 `t[0]` 또는 `t[1]`로 접근할 수 있습니다. `t[0]`은 가맹점명이므로 가맹점명 리스트에, `t[1]`은 이용금액이므로 이용금액 리스트에 추가하면 됩니다.

〈프로그램 흐름〉

4. 지출액 상위 10개의 가맹점명과 지출액을 각각 리스트에 저장하기

〈코드〉

```
4. top10_store, top10_amount = [], []
   for t in top10:
       top10_store.append(t[0])
       top10_amount.append(t[1])
```

가맹점 리스트는 `top10_store`, 지출액 리스트는 `top10_amount`로 만들었습니다. 코드를 보면 `top10_store`에는 가맹점명을 의미하는 `t[0]`이, `top10_amount`에는 지출액을 의미하는 `t[1]`이 추가됩니다.

이제 과정 5에서 그래프를 그릴 차례입니다. 앞에서 구한 두 리스트를 `plt.barh()`에 넣어 그래프를 그리면 됩니다.

〈프로그램 흐름〉

5. 가맹점명 리스트와 지출액 리스트로 막대그래프 그리기

〈코드〉

```
5. plt.barh(top10_store, top10_amount)
```

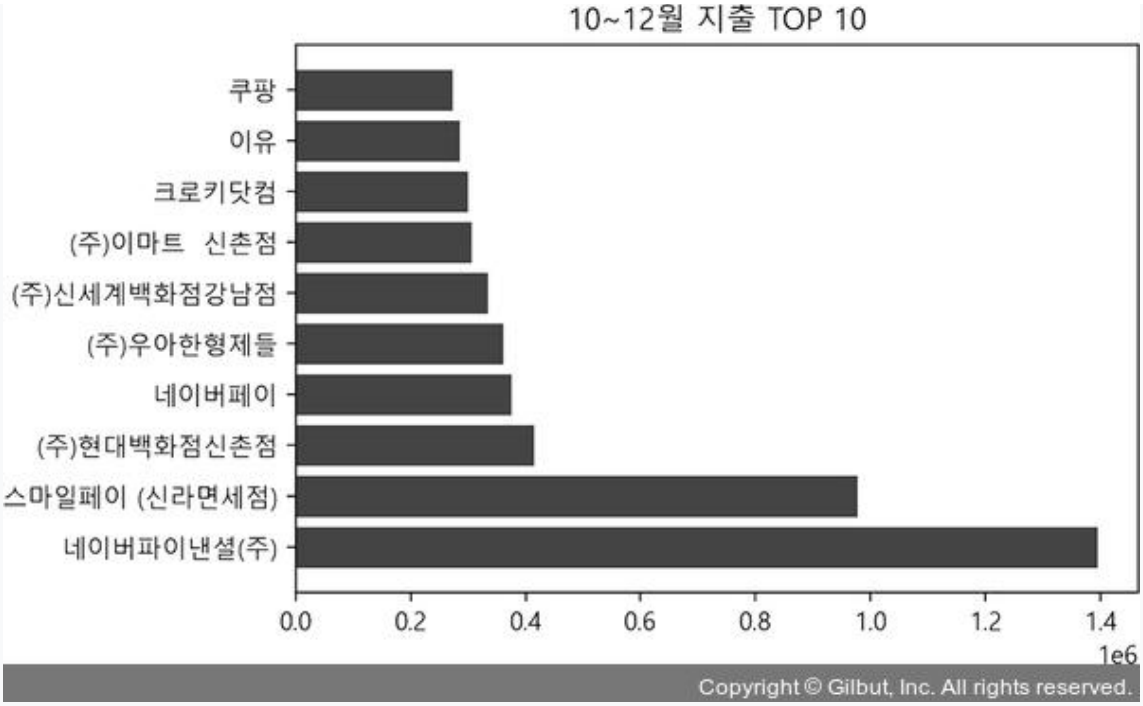
프로그램을 완성해 그래프를 그려 봅시다.

```
import operator
import matplotlib.pyplot as plt

spending = {}
for row in data:
    if row[-1] == '전표매입':
        store, payment = row[-4], int(row[-3])
        if store not in spending.keys():
            spending[store] = payment
        else:
            spending[store] += payment

top10 = sorted(spending.items(), key=operator.itemgetter(1), reverse=True)[:10]
top10_store, top10_amount = [], []
for t in top10:
    top10_store.append(t[0])
    top10_amount.append(t[1])
plt.rc('font', family='Malgun Gothic')
plt.title('10~12월 지출 TOP 10')
plt.barh(top10_store, top10_amount, color='b')
plt.show()
```

실행결과


10~12월 지출 TOP 10
쿠팡
이유
크로키닷컴
(주)이마트 신촌점
(주)신세계백화점강남점
(주)우아한형제들
네이버페이
(주)현대백화점신촌점
스마일페이 (신라면세점)
네이버파이낸셜(주)
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1e6
Copyright © Gilbut, Inc. All rights reserved.

처음에 손으로 그린 그래프와 비슷하게 그려집니다. 결과를 분석해 봅시다. 가장 많은 금액을 사용한 가맹점은 네이버파이낸셜(주)로, 약 140만 원을 지출했습니다. 두 번째로 많은 금액을 사용한 곳은 스마일페이 (신라면세점)입니다. 그런데 면세점은 해외여행을 갈 때만 이용할 수 있죠. 카드 내역 파일을 보면 11월에 신라면세점 이용이 집중됩니다. 11월에 해외여행을 다녀왔을 거라 추측할 수 있겠죠?

앞서 월별 지출액 그래프에서 11월이 다른 달보다 압도적으로 지출액이 많았습니다. 다른 달에는 없는 면세점 이용이 집중되면서 11월 지출액이 늘었다고 생각할 수 있습니다. 그리고 여행 가서 사용한 다른 부대비용도 있을 테고요. 지출 규모 3위와 6위는 백화점이네요. 배달음식 가맹점도 5위에 있습니다. 2019년 1회 평균 외식 비용 12,600원을 기준으로 계산했을 때 월 평균 9.6회 정도 배달음식을 주문했군요. 가장 불필요한 지출이라 생각하는데 지출 횟수가 많네요.

지금까지 데이터 파일을 읽어와 몇 가지 분석을 해 봤습니다. Part 1에서 배운 변수, 조건문, 반복문, 리스트, 딕셔너리를 실제로 어떻게 사용하는지도 배웠습니다. 본인의 카드 이용내역을 내려받아서 직접 분석해 보면 더 재밌는 결과를 얻을 수 있겠죠? 이 과정에서 파이썬 프로그래밍 실력도 키우고, 소비 생활도 점검할 수 있을 겁니다. 파이썬 데이터 분석 방법을 알았으니 다른 데이터 파일도 분석할 수 있습니다. 다음 장부터는 공공 데이터를 이용해 본격적으로 데이터 분석 프로젝트를 진행해 보겠습니다.

1분 퀴즈

2 다음은 card.csv 파일을 바탕으로 지출액 상위 10개 가맹점을 구하는 코드입니다. (가)~(마)에 들어갈 코드로 틀린 것을 고르세요.

```
import operator

spending = {}
for row in data:
    if row[-1] == '전표매입':
        store, payment = row[-4], int(row[-3])
        if store not in spending.keys():
            (가)
        else:
            (나)

top10 = sorted((다), key=(라), reverse=True)(마)
top10_store, top10_amount = [], []
for t in
    top10: top10_store.append(t[0])
    top10_amount.append(t[1])
```

① (가) - `spending[store] = payment`

② (나) - `spending[store] += payment`

③ (다) - `spending.values()`

④ (라) - `operator.itemgetter(1)`

⑤ (마) - `[:10]`

정답 및 해설

9

마무리

1 파일 읽어 들이기

```
import csv                # csv 라이브러리 포함하기

f = open('파일명.csv')    # 파일 열기
data = csv.reader(f)      # 파일 읽기
next(data)                # 표의 헤더 제거하기
data = list(data)         # 리스트로 변환하기
```

2 인코딩 옵션 추가하기

파일을 읽을 때 오류가 발생하면 `open()`에서 `encoding` 옵션을 추가한다.

```
# encoding 옵션으로 사용할 수 있는 값: utf8, cp949, euc-kr
f = open('파일명.csv' encoding = 'utf8')
```

3 데이터 타입 변환하기

CSV 파일에서 읽어 들인 데이터는 모두 문자로 인식되므로 연산이 필요하면 `int()`나 `float()`로 감싸서 숫자형으로 변환해야 한다.

Self Check

1 card.csv 파일은 10월부터 12월까지 3개월간의 카드 이용내역을 담은 파일입니다. 카드 이용내역에는 승인거래와 취소거래가 섞여 있습니다. 카드 이용내역 중 취소거래의 총액을 구하는 코드를 작성하세요.

힌트

1. 매입상태가 전표매입이 아닌 건은 모두 취소거래입니다.
2. 취소거래의 총액을 저장하는 변수가 있어야 합니다.

2 card.csv 파일의 카드 이용내역에서 월별로 백화점에서 사용한 금액을 꺾은선 그래프로 그리세요. 그래프 이름은 '10~12월 백화점 지출액'으로 합니다.

힌트

1. 그래프를 그리려면 10~12월 백화점 지출액을 하나의 리스트로 저장해야 합니다.
2. 백화점 지출액은 가맹점명에 '백화점' 키워드가 들어 있습니다.

정답 및 해설